

PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

SOFTWARE ENGINEERING – LEVEL 3 MINI-PROJECT (UE24CS341A)

Jackfruit Mini-Project · Phase 1 Submission · Semester V · Agile

Railway Reservation System

Train Scheduling, Seat Allocation and Ticket Management

Combined documentation: SRS + Software Architecture & Design + Software Test Plan

Group	Team 14 – Section B
Version / date	1.0 · 03-10-2026
Repository	https://github.com/be-hope/SE-Labs-PES1UG24CS111
Contents	Part A: SRS · Part B: Software Architecture & Design Specification (SAD) · Part C: Software Test Plan (STP)

Team member	SRN (USN)	PRN	Owned functionality	Module
Bhadra R S	PES1UG24CS111	PES1202400243	Registration, login and profile management	M1
Kshitij Prabhu	PES1UG24CS914	PES1202503636	Train search, schedule, route and fare display	M2
Catherine D	PES1UG24CS120	PES1202402174	Seat availability, booking, fare and PNR generation	M3
K Pranav	PES1UG25CS819	PES1202503156	Cancellation, waitlist and administration	M4

Document map

Part	Document	Highlights
A	Software Requirements Specification	25 functional, 7 non-functional and 6 security requirements with acceptance criteria · 13 use cases, 2 use-case diagrams · ER model · traceability matrix
B	Software Architecture & Design Specification	Layered architecture with 5 decision records · 12 UML figures · STRIDE model and RBAC matrix · API with data objects · error catalogue · UX
C	Software Test Plan	All 15 template sections · security validation (§5.1) · seed data · 72 test cases (unit, integration, system, security, non-functional) · requirement-to-test traceability

Technology: C / C++17 · SQLite · layered architecture · GoogleTest · Git/GitHub · Jenkins/Docker

Contents

Part A – Software Requirements Specification	3
1. Introduction	3
2. Overall description	4
3. External interface requirements	6
4. System features (functional requirements)	6
5. Non-functional requirements	9
6. Quality attributes and acceptance tests	11
7. System models and diagrams	11
8. Requirements traceability matrix (RTM)	15
Appendix A – Field layouts	17
Appendix B – Project plan (from Deliverable 2)	17
Part B – Software Architecture and Design Specification	19
1. Introduction	19
2. Document overview	19
3. Architecture	20
4. Design	28
4.2 UML sequence diagrams and behavioural models	29
4.3 API design	36
5. References	40
Part C – Software Test Plan	41
1. Introduction	41
2. Test items	41
3. Features to be tested	41
4. Features not to be tested	42
5. Test approach / strategy	42
6. Test environment	44
7. Test schedule	44
8. Test deliverables	44
9. Roles and responsibilities	45
10. Risks and mitigation	45
11. Assumptions and dependencies	45
12. Suspension and resumption criteria	45
13. Test case management and traceability	46
14. Test metrics and reporting	53
15. Approvals	53

Submission checklist

Requirement of the brief	Where
SRS in IEEE format, intro and descriptive sections	Part A §1–§3, §6
Functional and non-functional requirements, measurable	Part A §4 (25 FR) and §5 (7 NFR), each with an acceptance criterion and test cases
UML use-case diagram (at least 1)	Part A §7.1, Figures A-1 and A-2
Security objectives and requirements (at least 2 each)	Part A §5.1 – 4 objectives, 6 requirements
Test plan in IEEE format; sections 3, 4, 5 and 5.1	Part C §1–§15, §5.1
Traceability with the SRS; 7–10 test cases or more	Part A §8; Part C §13.2–§13.3 (72 test cases)
SAD: component diagram, pattern, traceability, security architecture	Part B §3.3, §3.5, §3.8, §3.9
SAD: at least 2 sequence diagrams, API design, error handling	Part B §4.2 (3 flows, 6 figures), §4.3, §4.4
Single combined PDF and Git repository	This document; repository on the cover page

Part A – Software Requirements Specification

Railway Reservation System – Train Scheduling, Seat Allocation and Ticket Management

Version	1.0
Authors	Team 14 – Bhadra R S, Kshitij Prabhu, Catherine D, K Pranav
Date	03-10-2026
Status	Draft 1.0 for evaluation (requirements baseline, Phase 1)
Repository	https://github.com/be-hope/SE-Labs-PES1UG24CS111

Revision history

Version	Date	Author	Change summary
0.9	15-09-2026	Team 14	Deliverable 2 baseline: FR-001 to FR-025, NFR-001 to NFR-007, SEC-001 to SEC-006, use-case list, business rules, field layouts, project plan.
1.0	03-10-2026	Team 14	Moved to the course SRS template. Added acceptance criteria and test references to every requirement, two use-case diagrams, ER model, use-case flows, security objectives and the full traceability matrix.

Approvals

Role	Name	Signature / date
Course faculty	Dr. Sapna V M, PES University	
Team 14	Bhadra R S · Kshitij Prabhu · Catherine D · K Pranav	

1. Introduction

1.1 Purpose

This SRS lists what the Railway Reservation System (RRS) must do, the limits it works within, and how each requirement will be checked. The architecture (Part B) and the test plan (Part C) refer back to the requirement IDs used here. It follows the course SRS template (IEEE 830 / ISO 29148) and builds on the Deliverable 1 synopsis and the Deliverable 2 requirements baseline.

1.2 Scope

RRS is a standalone, menu-driven C/C++ application for a railway ticketing office. Passengers search trains, see schedule, route, fare and seat availability, book tickets and get a PNR, and cancel with a calculated refund. Waitlisted tickets are promoted automatically. Reservation staff and administrators maintain trains, coaches and reservation records, and operations managers read booking and occupancy reports.

In scope	Out of scope
Passenger registration, login and profile Train search by source, destination and date Schedule, route and fare display Seat availability and seat allocation Booking and PNR generation Cancellation, refund, RAC / waitlist and ticket status Administration of trains, routes, schedules, coaches and reservations Operational reports (bookings, occupancy, train-wise counts) Role-based access control and audit logging	Physical railway operations, signalling and train movement Payment-gateway settlement (fare is calculated and accepted, not collected) Integration with live railway infrastructure or other reservation systems Graphical, web and mobile front-ends Partial cancellation of a multi-traveller PNR Networked, multi-user deployment

1.3 Audience

Course faculty and evaluators (start at \$4–\$8), the four developers (\$2.7, \$4, \$5.1) and testers (\$4–\$6, \$8).

1.4 Definitions and abbreviations

Term	Meaning
PNR	Passenger Name Record – the unique 10-digit reference of one reservation.
RAC	Reservation Against Cancellation – partly confirmed; the traveller is promoted to a full seat when one is released.
Waitlist (WL)	Ordered queue of tickets that could not be confirmed; position 1 is promoted first.

Term	Meaning
Class	Travel class of a coach: Sleeper (SL) or AC 3-Tier (3A) in the baseline data.
Coach / berth	A coach holds a set of seats or berths of one class; each berth has a unique number such as S1-07.
Schedule	A train running on one journey date.
Session	Logged-in period of a user, tied to one role, expiring after 15 minutes of inactivity.
Idempotency token (request_id)	Value sent with a booking so that a retry returns the original reservation instead of creating a second one.
Audit log	Append-only record of security-relevant events with time and actor.
RBAC	Role-based access control.
FR / NFR / SEC / BR / UC	Functional, non-functional, security requirement; business rule; use case.
AC / TC / RTM	Acceptance criterion; test case; requirements traceability matrix.

1.5 References

- Team 14, Deliverable 1 – Synopsis / Project Proposal, and Deliverable 2 – Requirements (baseline v0.9).
- Team 14, Architecture Selection and Justification (Layered Architecture), 15-09-2026.
- PES University, Software Engineering Level 3 Student Submission Guide and Jackfruit project guidelines, 2026.
- IEEE Std 830-1998; ISO/IEC/IEEE 29148:2018. OWASP Top 10 (2021) and MITRE CWE (used in §5.1).

1.6 Conventions

Every requirement has a unique tag (FR, NFR, SEC), uses “shall”, and has a measurable acceptance criterion with linked test cases. Priority: **High** – needed for the booking flow or for security/correctness; **Medium** – needed for the complete product but may move to a later sprint; **Low** – desirable.

2. Overall description

2.1 Product perspective

RRS is a self-contained application that replaces a manual, scattered reservation process. Users work through role-based console menus. All data sit in one embedded SQLite file (`rrs.db`); security events are appended to `audit.log`. There is no network interface and no dependency on an external reservation system or payment gateway.

External entity	Interaction with RRS
Passenger / Customer	Console menus: register, log in, search, book, cancel, check status
Reservation Staff	Console menus: book or correct reservations on behalf of passengers, manage waitlist
System Administrator	Console menus: maintain trains, routes, schedules, coaches and seats; manage records; reports
Railway Operations / Management	Console menus: read-only operational reports
Local file system	<code>rrs.db</code> (SQLite database) and <code>audit.log</code> (append-only)

2.2 Major product functions

Feature (owner)	Summary	Requirements
F1 Registration, login & profile (M1 – Bhadra R S)	Account creation with unique mobile/e-mail, login, profile maintenance, duplicate rejection	FR-001 – FR-004
F2 Train search, schedule & route (M2 – Kshitij Prabhu)	Search by source/destination/date, results, route and schedule view, per-class fare	FR-005 – FR-008
F3 Availability, booking, fare & PNR (M3 – Catherine D)	Real-time availability, traveller details, fare calculation, confirmation, seat allocation, PNR, history	FR-009 – FR-015
F4 Cancellation, waitlist & status (M4 – K Pranav)	Cancellation, refund slabs, waitlist/RAC with position, automatic promotion, PNR status	FR-016 – FR-020
F5 Administration, staff operations & reports (M4 – K Pranav)	Train/route/schedule/coach management, booking on behalf, record correction, reports	FR-021 – FR-025
Cross-cutting: security & quality	RBAC, hashing, validation, sessions, audit log; performance, reliability, usability, portability	SEC-001 – SEC-006, NFR-001 – NFR-007

2.3 User classes

User class	Privileges	Characteristics
Passenger / Customer	Register/login, search trains, view schedule/fare/availability, book, view PNR and history, cancel own tickets, maintain own profile	Non-technical; occasional-to-frequent use; menu-driven console
Reservation Staff	Create/update bookings on behalf of passengers, process cancellations, manage waitlist and reservation records	Trained user; frequent operational interaction; needs fast, error-tolerant screens
System Administrator	Add/update/delete train, route, schedule, coach and seat data; manage user and reservation records; reports	Highest privilege; trained; infrequent but critical operations
Railway Operations / Management	Read booking statistics, seat occupancy, schedules and reservation reports	Trained user; periodic, read-only reporting

2.4 Operating environment

- Ubuntu 22.04 LTS or later, or Windows 10/11 with MinGW-w64; ordinary desktop or laptop hardware.
- GCC / G++ 11 or later (C++17), built with CMake or Make, edited in Visual Studio Code.
- SQLite 3 for storage; Git + GitHub for version control; Jenkins in a Docker container for CI; SonarQube (or similar) for static analysis.

2.5 Design and implementation constraints

- C and C++ only, with a modular, object-oriented design (layered architecture, see Part B).
- Git/GitHub with individually attributable commits; Agile sprints as in the project plan (Appendix B).
- All database access goes through repository interfaces using bound parameters.
- GoogleTest for unit tests, gcov/lcov for coverage, GitHub Projects or Jira for the backlog.
- Passwords are never stored in plain text; privileged functions are protected by role checks (§5.1).

2.6 Assumptions and dependencies

- Train, route and initial schedule data are seeded and later maintained by an administrator.
- Staff, administrator and operations accounts are created by the seed script or by an administrator. Self-registration only creates PASSENGER accounts.
- Payment-gateway settlement is out of scope: a booking is confirmed once the traveller details are valid and the passenger accepts the calculated fare.
- The application runs on a single machine with read/write access to its working directory, one active session per console.
- The system clock is correct (sessions, refund slabs and the 90-day retention rule depend on it).

2.7 Business rules and domain parameters

ID	Business rule
BR-01	Only trains with available confirmed seats, RAC slots or waitlist capacity can accept new bookings.
BR-02	A passenger cannot modify passenger details after a ticket is confirmed and a PNR is generated; only cancellation is permitted.
BR-03	Only reservation staff or administrators can manually correct or override a reservation record.
BR-04	Only administrators can add, update or remove train, route and schedule data.
BR-05	A booking must include at least one valid passenger entry (maximum 6 per PNR).
BR-06	Cancellation and refund amounts follow the refund policy based on time-to-departure (slabs in Table 2.7-B).
BR-07	Waitlisted tickets are confirmed strictly in order of waitlist position when a confirmed seat becomes available.

Table 2.7-B – Domain parameters (baseline values). They are stored as configuration, editable by the administrator (FR-022), so they can change without code changes.

Parameter	Baseline value	Used by
Base rate per km	₹ 1.00 per km	FR-008, FR-011
Class multipliers	Sleeper (SL) 1.0 · AC 3-Tier (3A) 2.5	FR-008, FR-011
Travellers per PNR	1 to 6	FR-010, BR-05
Journey-date window	Today to today + 120 days	FR-005
Refund slabs	≥ 48 h before departure: 90 % · 12 h to < 48 h: 75 % · < 12 h: 50 % · WAITLISTED ticket: 100 % · after departure: not cancellable	FR-016, FR-017, BR-06

Parameter	Baseline value	Used by
RAC quota / waitlist capacity	Configurable per train and class; default RAC 0 and waitlist 20 tickets	FR-018, FR-019, FR-022
Session idle timeout	15 minutes	FR-002, SEC-004
Login lock-out	5 consecutive failures lock the account for 5 minutes	SEC-004
ID-proof retention	Cleared 90 days after the journey date	SEC-005

3. External interface requirements

3.1 User interface

A numbered, role-based console menu. After login the user sees only the menu of his or her role. Conventions on every screen: invalid choices answer “Invalid choice” and redisplay the menu; option 0 goes back; messages use the format `ERROR [Exxxx] : reason. Action. or OK: result.` (codes in SAD §4.4); the password and ID-proof number are masked or hidden; each input is validated immediately with a field-level message.

```
=====
RAILWAY RESERVATION SYSTEM v1.0          [Passenger]
=====
1. Search trains           5. My bookings
2. Schedule, route & fare 6. PNR status
3. Check seat availability 7. Cancel ticket
4. Book ticket            8. My profile
0. Logout
-----
Enter your choice >
```

3.2 Hardware, software and communication interfaces

Hardware: a keyboard and a text display of at least 80 × 24 characters. **Communications:** none; the baseline system is standalone (a networked version would need TLS 1.2 or later and a new security review).

Component	Version	Purpose	Data exchanged
SQLite	3.x (amalgamation vendored in the repository)	Embedded relational store	Train, route, schedule, account, reservation records via the C API with bound parameters
C++17 standard library	GCC 11+ / MinGW-w64	Containers, strings, file streams, chrono	In-memory structures; audit.log lines
GoogleTest, gcov / lcov	1.14+ / GCC	Development only	Unit tests and coverage reports

4. System features (functional requirements)

All requirements in this section are of type *Functional*. The test-case column points to the Software Test Plan (Part C).

4.1 Passenger registration, login and profile

Owner M1 – Bhadra R S. Priority High. Register → system validates and stores a hashed password; Login → role-bound session or one generic error; My profile → edit and save.

Req ID	Requirement (the system shall ...)	Pri.	Acceptance criterion and verification	Test cases	Depends on
FR-001	The system shall allow a new passenger to register an account using a unique mobile number or email ID. Source: Passenger	High	AC-001: A valid, unused 10-digit mobile number or e-mail plus a policy-compliant password (≥ 8 characters, at least one letter and one digit) creates exactly one account with role PASSENGER and a unique passenger_id. Verification: Functional test	UT-AUTH-01, UT-AUTH-02, IT-AUTH-01, ST-AUTH-01	SEC-001, SEC-003
FR-002	The system shall allow a registered passenger to log in using valid credentials. Source: Passenger	High	AC-002: Correct credentials start a session bound to the account role with a 15-minute idle timeout; wrong credentials return one generic error message and create no session. Verification: Functional test	UT-AUTH-03, UT-AUTH-04, ST-AUTH-01, ST-AUTH-02	SEC-004

Req ID	Requirement (the system shall ...)	Pri.	Acceptance criterion and verification	Test cases	Depends on
FR-003	The system shall allow an authenticated passenger to create and update their profile (name, age, gender, contact details, ID-proof type and number). Source: Passenger	High	AC-003: Name (1–100 chars), age (1–120), gender (M/F/O) and ID proof (type, number ≤ 30 chars) are saved; the next profile view shows the new values; out-of-range values are rejected with a field-level message. Verification: Functional test	IT-AUTH-03, UT-AUTH-05, ST-AUTH-01	FR-002, SEC-005
FR-004	The system shall reject registration attempts that use a mobile number/email already associated with an existing account. Source: Ticketing Ops	High	AC-004: A second registration with an existing mobile number or e-mail is rejected with ERR-DUP-USER (E1002); the account count is unchanged. Verification: Negative test	IT-AUTH-02	FR-001

4.2 Train search, schedule and route

Owner M2 – Kshitij Prabhu. Priority High. Source, destination and date → matching trains; select a train → ordered route and fare of every class.

Req ID	Requirement (the system shall ...)	Pri.	Acceptance criterion and verification	Test cases	Depends on
FR-005	The system shall allow a passenger to search for trains by source station, destination station and journey date. Source: Passenger	High	AC-005: For a source, destination and journey date (today to today + 120 days) all active trains that visit the source before the destination and run on that date are listed and no others; source = destination, an unknown station code or an out-of-range date is rejected. Verification: Functional test	UT-TRN-01, UT-TRN-02, IT-TRN-01, IT-TRN-03, ST-TRN-01	SEC-003
FR-006	The system shall display all matching trains with train number, name, departure time, arrival time and duration. Source: Passenger	High	AC-006: Every result row shows train number, name, departure time at the source, arrival time at the destination and duration (hh:mm, correct across midnight). Verification: Functional test	IT-TRN-01, IT-TRN-02, UT-TRN-03, ST-TRN-01	FR-005
FR-007	The system shall allow a passenger to view the full schedule and route (ordered list of stopping stations) for a selected train. Source: Passenger	Medium	AC-007: The route view lists every stop in order with arrival time, departure time and cumulative distance in km. Verification: Functional test	IT-TRN-04, ST-TRN-01	FR-005
FR-008	The system shall display fare information for each available class for the selected train and journey. Source: Passenger	High	AC-008: A fare is shown for every class configured on the train: fare = distance (km) × ₹1.00 per km × class multiplier (Sleeper 1.0, AC 3-Tier 2.5), rounded to 2 decimals. Verification: Functional test	UT-TRN-04, IT-TRN-05, ST-TRN-01	FR-011

4.3 Seat availability, booking, fare and PNR

Owner M3 – Catherine D. Priority High. Select train, date and class → seats free, RAC slots and waitlist length; enter 1–6 travellers → total fare; accept → seats, PNR and ticket.

Req ID	Requirement (the system shall ...)	Pri.	Acceptance criterion and verification	Test cases	Depends on
FR-009	The system shall display real-time seat availability for a selected train, date and class. Source: Passenger	High	AC-009: The screen shows confirmed seats free, RAC slots free and waitlist length for the train, date and class; the numbers equal the stored values at query time and change immediately after a booking or cancellation. Verification: Functional test	UT-BKG-05, IT-BKG-01, ST-BKG-01, PERF-02	FR-005
FR-010	The system shall allow a passenger to enter passenger details (name, age, gender, ID proof) for each traveller in a booking. Source: Passenger	High	AC-010: 1 to 6 travellers per booking; each has name (1–100 chars), age (1–120), gender (M/F/O) and ID-proof number; any invalid field blocks the booking with a field-level message. Verification: Functional test	UT-BKG-02, IT-BKG-03, ST-BKG-01, ST-BKG-02	SEC-003

Req ID	Requirement (the system shall ...)	Pri.	Acceptance criterion and verification	Test cases	Depends on
FR-011	The system shall calculate the total fare from the base fare, class and number of passengers before booking confirmation. Source: Passenger	High	AC-011: Total fare = base fare × class multiplier × number of travellers and is displayed before confirmation; for example 350 km, AC 3-Tier, 2 travellers = ₹1,750.00. Verification: Unit + system test	UT-TRN-04, UT-BKG-01, ST-BKG-01, ST-BKG-02	FR-008
FR-012	The system shall confirm a reservation only after all mandatory passenger details are valid and the calculated fare is accepted (payment-gateway settlement is out of scope). Source: Passenger	High	AC-012: A reservation record is created only when every traveller validates and the passenger accepts the displayed fare; otherwise no record is written and seat counts are unchanged. Verification: Negative + positive test	IT-BKG-02, IT-BKG-03, IT-BKG-04, ST-BKG-01, ST-BKG-02	FR-010, FR-011
FR-013	The system shall allocate a specific seat/berth number to each confirmed passenger at the time of booking confirmation. Source: Ticketing Ops	High	AC-013: Each confirmed traveller receives a seat number (coach-seat, e.g. S1-07); no seat is allocated to two active tickets on the same train, date and class. Verification: Integration test	UT-BKG-04, IT-BKG-02, IT-BKG-05, ST-BKG-01	FR-009
FR-014	The system shall generate a unique PNR number for every successfully created reservation. Source: Ticketing Ops	High	AC-014: Each reservation receives a 10-digit numeric PNR (first digit 1-9); 10,000 generated PNRs contain no duplicate. Verification: Integration test	UT-BKG-03, IT-BKG-02, IT-BKG-06, ST-BKG-01	FR-012
FR-015	The system shall maintain a booking history accessible to the passenger, showing past and upcoming reservations. Source: Passenger	Medium	AC-015: The passenger sees all own reservations (any status) ordered by journey date and labelled Upcoming or Past; reservations of other passengers are never shown. Verification: Functional test	IT-BKG-07	SEC-002

4.4 Cancellation, waitlist and ticket status

Owner M4 – K Pranav. Priority High. PNR + Cancel → ownership and time check, refund, waitlist promotion; PNR + Status → Confirmed / RAC / Waitlisted (position) / Cancelled.

Req ID	Requirement (the system shall ...)	Pri.	Acceptance criterion and verification	Test cases	Depends on
FR-016	The system shall allow a passenger to cancel a booked ticket before the journey date. Source: Passenger	High	AC-016: A CONFIRMED, RAC or WAITLISTED ticket owned by the passenger can be cancelled until the departure time at the boarding station; later attempts are rejected with ERR-CANCEL-DEPARTED (E4003). Verification: Functional test	UT-CAN-02, IT-CAN-01, IT-CAN-02, ST-CAN-01	SEC-002
FR-017	The system shall calculate the applicable refund amount based on the defined cancellation policy and time of cancellation. Source: Ticketing Ops	High	AC-017: Refund = fare × slab: 90 % if cancelled ≥ 48 h before departure; 75 % from 12 h up to < 48 h; 50 % if < 12 h; 100 % for WAITLISTED tickets. Boundary values (48 h, 12 h) behave as stated. Verification: Unit test	UT-CAN-01, IT-CAN-01, IT-CAN-04, ST-CAN-01	FR-016, BR-06
FR-018	The system shall place a passenger on a waitlist, with a visible waitlist position, when no confirmed seats are available. Source: Passenger	High	AC-018: When no confirmed (and no RAC) capacity remains and the waitlist is not full, the ticket is WAITLISTED with position = number of waitlisted tickets ahead + 1, shown to the passenger; a full waitlist rejects the booking with ERR-NO-CAPACITY (E3001). Verification: Functional test	UT-CAN-03, IT-CAN-04, IT-CAN-05, ST-CAN-01	FR-012, BR-01
FR-019	The system shall automatically promote the next waitlisted ticket to confirmed status when a confirmed seat becomes available due to cancellation. Source: Passenger	High	AC-019: When a seat is released the head-of-queue ticket is promoted in the same transaction, receives seat numbers and all later positions decrease by 1; promotion never skips the head of the queue (BR-07). Verification: Integration test	IT-CAN-03, UT-CAN-04, IT-CAN-05, ST-CAN-01	FR-016, BR-07

Req ID	Requirement (the system shall ...)	Pri.	Acceptance criterion and verification	Test cases	Depends on
FR-020	The system shall allow a passenger to check the current status of a ticket (Confirmed, RAC, Waitlisted, Cancelled) using the PNR number. Source: Passenger	High	AC-020: The status of a PNR is shown as Confirmed, RAC, Waitlisted (with position) or Cancelled to its owner and to staff/administrator; an unknown PNR returns ERR-PNR-NOT-FOUND (E4001). Verification: Functional test	IT-CAN-05, IT-CAN-06, UT-CAN-05, ST-CAN-01	FR-014, SEC-002

4.5 Administration, staff operations and reporting

Owner M4 – K Pranav. Priority High. Administrator manages trains, routes, schedules, coaches; staff book or correct on behalf of passengers; operations and administrators view reports.

Req ID	Requirement (the system shall ...)	Pri.	Acceptance criterion and verification	Test cases	Depends on
FR-021	The system shall allow an administrator to add, update or remove train, route and schedule information. Source: System Administrator	High	AC-021: Added trains, routes and schedules appear in search; updates are reflected immediately; removal of a train with active future reservations is rejected with ERR-TRAIN-IN-USE (E5001), otherwise the train is deactivated. Verification: Functional test	IT-ADM-01, IT-ADM-02, IT-ADM-03, UT-ADM-01, ST-ADM-01	SEC-002, BR-04
FR-022	The system shall allow an administrator to manage coach and seat/berth configuration for each train. Source: System Administrator	Medium	AC-022: Coaches, seats per coach, RAC quota and waitlist capacity can be set per train and class; capacity = coaches × seats per coach; reducing capacity below the seats already booked is rejected with ERR-CAPACITY-BELOW-BOOKED (E5002). Verification: Functional test	IT-ADM-04	FR-021
FR-023	The system shall allow reservation staff to create or update a booking on behalf of a passenger. Source: Reservation Staff	Medium	AC-023: A booking made by staff follows every passenger-booking rule (validation, fare, seat, PNR) and records booked_by = the staff account. Verification: Functional test	IT-STF-01, ST-ADM-01	FR-012, SEC-002
FR-024	The system shall allow an administrator or reservation staff to manually correct/cancel a reservation record when required. Source: Reservation Staff / Admin	Medium	AC-024: Staff or administrator can correct traveller details or cancel a reservation; each change writes an audit entry (actor, time, old value, new value); a passenger session cannot perform the operation. Verification: Functional test	IT-STF-02, SEC-T02	SEC-002, SEC-006, BR-03
FR-025	The system shall allow an administrator or railway-operations user to generate reports on booking statistics, seat occupancy and train-wise reservation counts. Source: Railway Operations / Management	Medium	AC-025: Booking statistics (by status), seat occupancy (%) and train-wise reservation counts are produced for a date range and equal the figures obtained by direct database queries. Verification: Functional test	IT-RPT-01, IT-RPT-02, ST-ADM-01	SEC-002

5. Non-functional requirements

Targets that depend on load or environment state it explicitly.

Req ID	Requirement	Category	Pri.	Measurable acceptance criterion / test
NFR-001	95% of train search queries shall return results within 2 seconds under the defined test load.	Performance	High	Test load: 200 trains, 40 stations, 20,000 reservations. 100 sequential search queries; at least 95 complete in ≤ 2.0 s (report P50 / P95 / max). Test: PERF-01, PERF-02.
NFR-002	The application shall be available at least 99% during the defined operating window, excluding planned maintenance.	Availability	High	Operating window 06:00–23:00 daily. A 1,000-operation soak run finishes without a crash and with ≥ 99 % of operations completing normally; operational log reviewed. Test: REL-01.
NFR-003	A new passenger shall be able to complete a basic booking in no more than 6 primary menu steps.	Usability	Medium	Steps: Login → Search → Select train/class → Enter travellers → Confirm fare → View ticket (= 6). Walkthrough with 5 first-time users: ≥ 4 of 5 complete unaided in ≤ 6 steps. Test: ST-BKG-01, USB-01.

Req ID	Requirement	Category	Pri.	Measurable acceptance criterion / test
NFR-004	A reservation shall not be created twice if a passenger retries after a booking-confirmation timeout.	Reliability	High	A retry carrying the same request token returns the original PNR and the reservation count stays 1, also after a process restart. Test: IT-BKG-06, REL-02.
NFR-005	Business logic (search, booking, cancellation, waitlist) shall be separated into identifiable modules with automated unit tests.	Maintainability	Medium	One header/source pair per component; no UI → database calls (review checklist 100 %); gcov line coverage ≥ 80 % and branch coverage ≥ 70 % on the business layer. Test: MNT-01.
NFR-006	Core services (seat allocation, fare calculation, PNR generation) shall expose testable interfaces independent of file/database I/O.	Testability	Medium	Unit suites for FareCalculator, SeatAllocator and PnrGenerator pass with no file or database opened (verified with strace). Test: TST-01, UT-BKG-01/03/04.
NFR-007	The core C/C++ modules shall compile and run without modification using the project's specified GCC/G++ toolchain.	Portability	Medium	Clean build with g++ -std=c++17 -Wall -Wextra -Werror on Linux (GCC 11+) and Windows (MinGW-w64); unit suite passes on both. Test: PORT-01.

5.1 Security

5.1.1 Security objectives

ID	Objective	Description	Satisfied by
SO-1	Confidentiality of credentials and identity data	Passwords and passengers' identity-proof data must never be disclosed through storage, screens or logs.	SEC-001, SEC-005
SO-2	Access-control integrity	Only an authenticated user holding the proper role may execute a privileged operation; sessions cannot be forged, replayed after expiry or used to escalate privilege.	SEC-002, SEC-004
SO-3	Integrity of reservation data and input trust	Untrusted input must not corrupt reservation data or cause memory-safety faults; every value is validated before use and all database access is parameterised.	SEC-003
SO-4	Accountability	Security-relevant events must be recorded in a tamper-evident, append-only audit trail so that misuse can be detected and attributed.	SEC-006

5.1.2 Security requirements

Req ID	Requirement (the system shall ...)	Pri.	Acceptance criterion and validation	Test cases	Threat / reference
SEC-001	Passwords/credentials shall not be stored in plaintext.	High	Passwords are stored only as salted PBKDF2-HMAC-SHA256 hashes (≥ 10,000 iterations, 16-byte random salt). A text search of the database and logs for a known password finds nothing; two accounts with the same password have different hashes. Validation: Code/configuration review	UT-AUTH-02, SEC-T01	Credential disclosure – CWE-256, CWE-916; OWASP A02
SEC-002	Role-based access shall prevent passengers from accessing reservation-staff or administrator functions.	High	Each operation is permitted only to the roles of the permission matrix (SAD §3.9.3); every other role receives ERR-FORBIDDEN (E1006) and no data changes. A passenger can act only on own reservations. Validation: Authorization tests	IT-CAN-02, SEC-T02, SEC-T05, SEC-T08	Privilege escalation, IDOR – CWE-285, CWE-639; OWASP A01
SEC-003	The system shall validate and sanitize all user-supplied input (passenger details, search parameters) before processing.	High	Every string is length-, format- and range-checked before use; SQL uses bound parameters only. 1,000-character, control-character, SQL-meta (' OR 1=1 --) and %s%n inputs end in a clean validation error – no crash and an AddressSanitizer/UBSan-clean run. Validation: Negative/security tests	UT-AUTH-01, UT-TRN-01, UT-BKG-02, UT-CAN-05, SEC-T03, SEC-T08	Injection, buffer overflow – CWE-20, CWE-89, CWE-120; OWASP A03
SEC-004	The system shall reject authenticated operations when the session/credentials are invalid or expired.	High	A session expires after 15 minutes of inactivity; a missing, expired or altered session is rejected with ERR-SESSION-EXPIRED (E1005); 5 consecutive failed logins lock the account for 5 minutes (E1004). Validation: Security test	UT-AUTH-04, ST-AUTH-02, SEC-T04, SEC-T06	Session misuse, brute force – CWE-613, CWE-307; OWASP A07
SEC-005	Sensitive passenger identification data shall not be retained beyond what is necessary for reservation processing.	Medium	ID-proof numbers are displayed masked (last 4 characters only), never written to logs, and cleared 90 days after the journey date by the retention routine. Validation: Design review	SEC-T07	Excess data retention – CWE-359; data minimisation

Req ID	Requirement (the system shall ...)	Pri.	Acceptance criterion and validation	Test cases	Threat / reference
SEC-006	Security-relevant events (failed logins, unauthorized attempts to access admin functions) shall be logged.	Medium	Failed logins, account lock-outs, unauthorised attempts and privileged changes are appended to the audit log with timestamp, actor and event type within the same operation; the log never contains passwords or ID numbers. Validation: System test	SEC-T05, SEC-T06	Insufficient logging – CWE-778; OWASP A09

5.2 Other requirements

- **Database:** one SQLite file (rrs.db); referential integrity and UNIQUE constraints on accounts, PNRs and request tokens are enforced by the database.
- **Data retention:** ID-proof numbers are cleared 90 days after the journey (SEC-005); reservation and audit records are kept for the life of the database.
- **Privacy:** only the personal data needed for a reservation is collected (data minimisation, in line with India's DPDP Act 2023).
- **Localisation:** English interface, amounts in Indian rupees (₹), 24-hour time, dates as DD-MM-YYYY.
- **Safety:** software only; no physical equipment is controlled.

6. Quality attributes and acceptance tests

Exit criteria for acceptance: all High-priority functional requirements are implemented and verified; no critical NFR or security failure is open; the RTM (§8) shows at least one passing test for every requirement; unit-test coverage meets NFR-005. The way each category is validated (functional black-box tests, authorisation and penetration-style tests, load and soak runs, usability walkthrough, retry/fault-injection, unit tests with coverage) is detailed in Part C.

Acceptance suite	Scope	Test cases (STP §13)
Registration & login	FR-001 – FR-004, SEC-004	UT/IT/ST-AUTH-xx
Search & schedule	FR-005 – FR-008	UT/IT/ST-TRN-xx
Booking, fare & PNR	FR-009 – FR-015	UT/IT/ST-BKG-xx
Cancellation & waitlist	FR-016 – FR-020	UT/IT/ST-CAN-xx
Administration & reports	FR-021 – FR-025	IT-ADM/STF/RPT-xx, ST-ADM-01
Security	SEC-001 – SEC-006	SEC-T01 ... SEC-T08
Performance, reliability, usability, portability	NFR-001 – NFR-007	PERF, REL, USB, MNT, TST, PORT

7. System models and diagrams

7.1 UML use-case diagrams

Diagram 1 shows the passenger use cases; diagram 2 shows staff, administrator, operations and the automatic waitlist process. UC-01 to UC-11 come from Deliverable 2; UC-12 and UC-13 were added for FR-003 and FR-015.

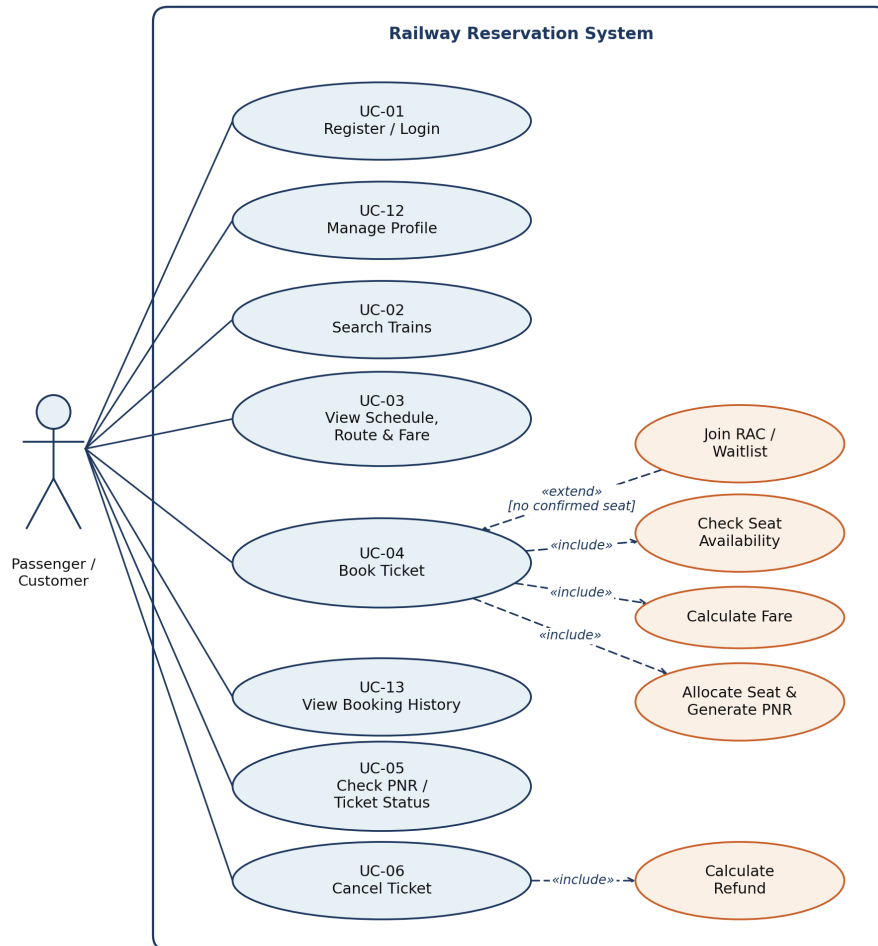


Figure A-1 – Use-case diagram 1: Passenger (UC-01 to UC-06, UC-12, UC-13)

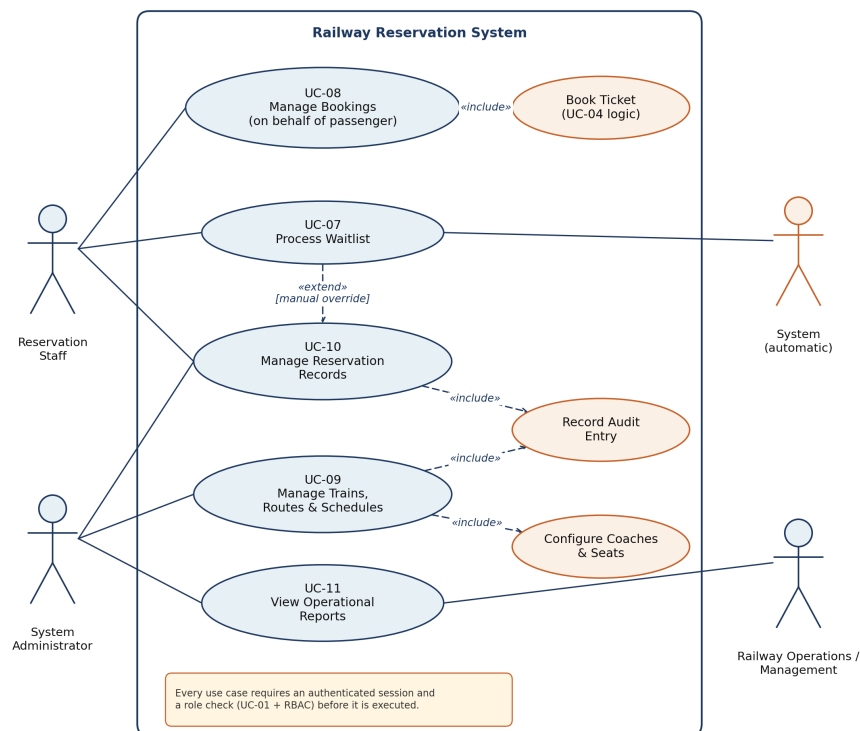


Figure A-2 – Use-case diagram 2: Reservation Staff, System Administrator, Railway Operations and System (UC-07 to UC-11). The administrator can also run UC-11 (FR-025, SAD §3.9.3).

Use-case catalogue

ID	Use case	Primary actor	Precondition	Outcome	Requirements
UC-01	Register / Login	Passenger	Application available	Authenticated session	FR-001, FR-002, FR-004
UC-02	Search Trains	Passenger	None	Matching trains displayed	FR-005, FR-006
UC-03	View Schedule, Route & Fare	Passenger	Train selected	Schedule/route/fare displayed	FR-007, FR-008
UC-04	Book Ticket	Passenger	Logged in; valid passenger details	Reservation created; PNR generated	FR-009 – FR-014
UC-05	Check PNR / Ticket Status	Passenger	PNR exists	Current status displayed	FR-020
UC-06	Cancel Ticket	Passenger	Valid, active booking exists	Ticket cancelled; refund calculated	FR-016, FR-017
UC-07	Process Waitlist	System / Reservation Staff	Cancellation or new capacity occurs	Waitlist re-ordered / updated	FR-018, FR-019
UC-08	Manage Bookings	Reservation Staff	Staff authenticated	Booking created/updated on passenger's behalf	FR-023
UC-09	Manage Trains, Routes & Schedules	System Administrator	Admin authenticated	Train/route/schedule data updated	FR-021, FR-022
UC-10	Manage Reservation Records	System Administrator / Staff	Authenticated	Reservation records updated/corrected	FR-024
UC-11	View Operational Reports	Railway Operations / Management; System Administrator	Authenticated	Report displayed	FR-025
UC-12	Manage Profile *	Passenger	Logged in	Profile created / updated	FR-003
UC-13	View Booking History *	Passenger	Logged in	Own reservations listed	FR-015

UC-04 Book Ticket – flow

Preconditions	1. The passenger is logged in (valid session). 2. The train, journey date and class exist and the train is active.
Postconditions	A reservation with a unique PNR and status CONFIRMED, RAC or WAITLISTED is stored; seats are allocated for CONFIRMED tickets; an audit event is written.
Main success scenario	1. The passenger searches and selects a train, date and class. 2. The system shows seat availability and the fare per class. 3. The passenger enters 1–6 travellers. 4. The system validates every field. 5. The system calculates and displays the total fare. 6. The passenger accepts the fare. 7. The system checks capacity, allocates seats, generates a unique PNR and stores the reservation in one transaction. 8. The system displays the ticket (PNR, status, seats, fare).
Alternate flows	4a. A traveller field is invalid → the system shows a field-level message and repeats step 3. 6a. The passenger rejects the fare → no reservation is created. 7a. No confirmed seat is free → the ticket becomes RAC (if quota free) or WAITLISTED with its position. 7b. No seat, RAC slot or waitlist room → ERR-NO-CAPACITY (E3001); nothing is stored. 7c. The same request token is received again → the original PNR is returned (no duplicate).
Related requirements	FR-009 ... FR-014, FR-018, NFR-003, NFR-004, SEC-002, SEC-003, BR-01, BR-05

UC-06 Cancel Ticket – flow

Preconditions	1. The passenger is logged in. 2. The PNR exists, belongs to the passenger and is not CANCELLED.
Postconditions	The ticket is CANCELLED; the refund is computed and stored; released seats are re-offered to the waitlist; an audit event is written.
Main success scenario	1. The passenger enters the PNR and chooses Cancel. 2. The system validates the PNR format and the ownership. 3. The system checks that the departure time has not passed. 4. The system calculates the refund from the slab of Table 2.7-B. 5. The system marks the ticket CANCELLED and releases its seats. 6. The system promotes waiting tickets in strict order (UC-07). 7. The system shows the cancellation receipt with the refund.

Alternate flows	2a. PNR unknown → ERR-PNR-NOT-FOUND (E4001). 2b. PNR belongs to another passenger → ERR-FORBIDDEN (E1006) and an audit entry. 3a. Train already departed → ERR-CANCEL-DEPARTED (E4003). 3b. Ticket already cancelled → ERR-ALREADY-CANCELLED (E4002).
Related requirements	FR-016, FR-017, FR-019, SEC-002, BR-06, BR-07

7.2 Domain (entity-relationship) model

Stored data with primary keys, foreign keys and the constraints that keep accounts, PNRs and booking tokens unique. Field lengths are in Appendix A.

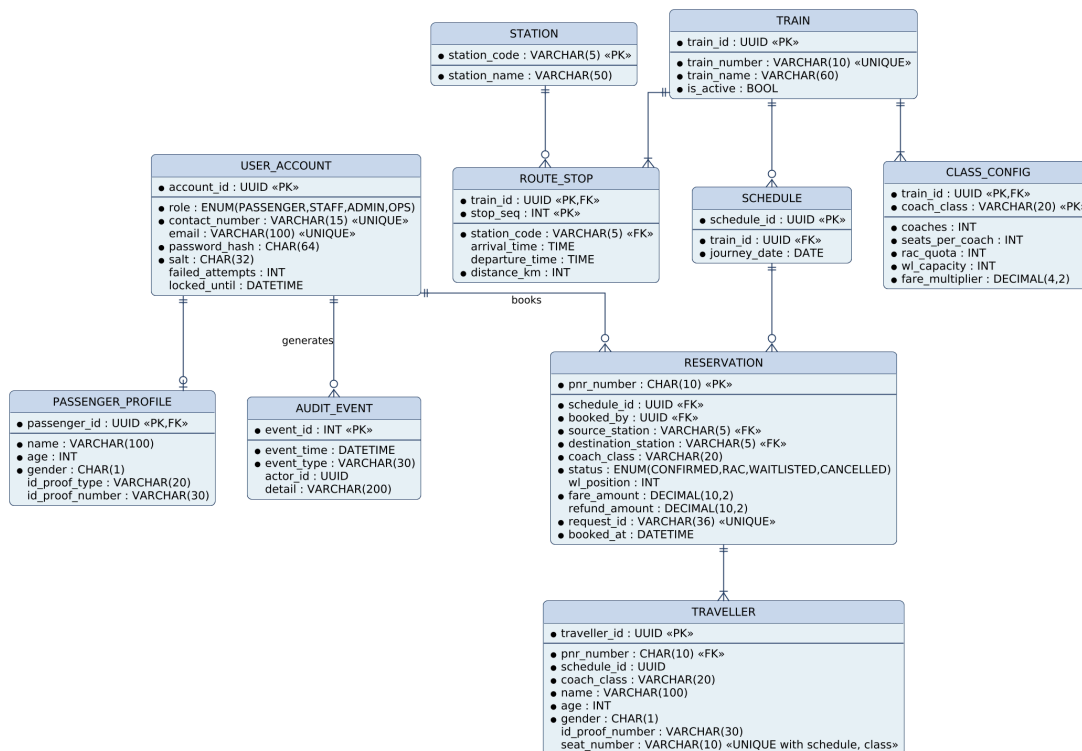


Figure A-3 – Entity-relationship model of RRS

8. Requirements traceability matrix (RTM)

Each requirement is linked to its SRS section, the architecture component (SAD §3.4, C1–C8), the design element (SAD §4), the planned source file and the verifying test cases (STP §13). Status: N = not implemented, P = partly implemented, A = accepted (implemented and all linked tests passed). The status column is updated at the end of every sprint.

Req ID	Requirement (brief)	SRS §	Arch. ref	Design reference	Code / file reference (planned)	Unit / integration TC	System / NFR / security TC	St.
FR-001	Register with unique mobile / e-mail	4.1	C2	AuthService.registerPassenger · SD-03 · CD-01 UserAccount	auth/auth_service.cpp	UT-AUTH-01, UT-AUTH-02, IT-AUTH-01	ST-AUTH-01	N
FR-002	Log in with valid credentials	4.1	C2	AuthService.login · Session · SD-03	auth/auth_service.cpp, auth/session.cpp	UT-AUTH-03, UT-AUTH-04	ST-AUTH-01, ST-AUTH-02	N
FR-003	Create and update profile	4.1	C2	AuthService.updateProfile · PassengerProfile	auth/auth_service.cpp	IT-AUTH-03, UT-AUTH-05	ST-AUTH-01	N
FR-004	Reject duplicate registration	4.1	C2, C7	UNIQUE(contact_number) · ERR-DUP-USER	auth/auth_service.cpp, dal/sqlite_user_repo.cpp	IT-AUTH-02	–	N
FR-005	Search trains by source, destination, date	4.2	C3	TrainService.searchTrains · Train.servesLeg	train/train_service.cpp	UT-TRN-01, UT-TRN-02, IT-TRN-01, IT-TRN-03	ST-TRN-01	N
FR-006	Show number, name, times, duration	4.2	C3	TrainSummary · duration calculation	train/train_service.cpp	IT-TRN-01, IT-TRN-02, UT-TRN-03	ST-TRN-01	N
FR-007	View schedule and route of a train	4.2	C3	TrainService.getRoute · RouteStop	train/train_service.cpp	IT-TRN-04	ST-TRN-01	N
FR-008	Show fare for each class	4.2	C3, C4	TrainService.getFareQuotes · FareCalculator	train/train_service.cpp, booking/fare_calculator.cpp	UT-TRN-04, IT-TRN-05	ST-TRN-01	N
FR-009	Show real-time seat availability	4.3	C4, C7	BookingService.checkAvailability	booking/booking_service.cpp	UT-BKG-05, IT-BKG-01	ST-BKG-01, PERF-02	N
FR-010	Enter details of each traveller	4.3	C4, C6	InputValidator.validateTraveller · Traveller	security/input_validator.cpp	UT-BKG-02, IT-BKG-03	ST-BKG-01, ST-BKG-02	N
FR-011	Calculate total fare before confirmation	4.3	C4	FareCalculator.totalFare · SD-01 steps 4-7	booking/fare_calculator.cpp	UT-TRN-04, UT-BKG-01	ST-BKG-01, ST-BKG-02	N
FR-012	Confirm only valid, fare-accepted bookings	4.3	C4	BookingService.createBooking · SD-01 · AD-01	booking/booking_service.cpp	IT-BKG-02, IT-BKG-03, IT-BKG-04	ST-BKG-01, ST-BKG-02	N
FR-013	Allocate a seat / berth per passenger	4.3	C4	SeatAllocator.allocate · SD-01	booking/seat_allocator.cpp	UT-BKG-04, IT-BKG-02, IT-BKG-05	ST-BKG-01	N
FR-014	Generate a unique PNR	4.3	C4	PnrGenerator.generate · SD-01	booking/pnr_generator.cpp	UT-BKG-03, IT-BKG-02, IT-BKG-06	ST-BKG-01	N
FR-015	Booking history for the passenger	4.3	C4, C7	BookingService.getBookingHistory	booking/booking_service.cpp	IT-BKG-07	–	N
FR-016	Cancel a ticket before the journey	4.4	C5	CancellationService.cancelTicket · SD-02	cancel/cancellation_service.cpp	UT-CAN-02, IT-CAN-01, IT-CAN-02	ST-CAN-01	N
FR-017	Compute refund by cancellation policy	4.4	C5	RefundCalculator.refund · SRS Table 2.7-B	cancel/refund_calculator.cpp	UT-CAN-01, IT-CAN-01, IT-CAN-04	ST-CAN-01	N
FR-018	Waitlist with visible position	4.4	C4, C5	BookingService status decision · SM-01	booking/booking_service.cpp	UT-CAN-03, IT-CAN-04, IT-CAN-05	ST-CAN-01	N
FR-019	Automatic waitlist promotion	4.4	C5	WaitlistManager.promote · SD-02 · SM-01	cancel/waitlist_manager.cpp	IT-CAN-03, UT-CAN-04, IT-CAN-05	ST-CAN-01	N
FR-020	Ticket status by PNR	4.4	C5	CancellationService.getTicketStatus · SM-01	cancel/cancellation_service.cpp	IT-CAN-05, IT-CAN-06, UT-CAN-05	ST-CAN-01	N
FR-021	Admin: manage trains, routes, schedules	4.5	C5	AdminService.add/update/removeTrain	admin/admin_service.cpp	IT-ADM-01, IT-ADM-02, IT-ADM-03, UT-ADM-01	ST-ADM-01	N
FR-022	Admin: coach and seat configuration	4.5	C5	AdminService.configureCoaches · ClassConfig	admin/admin_service.cpp	IT-ADM-04	–	N
FR-023	Staff: book on behalf of a passenger	4.5	C5, C4	AdminService.bookOnBehalf → BookingService	admin/admin_service.cpp	IT-STF-01	ST-ADM-01	N

Req ID	Requirement (brief)	SRS §	Arch. ref	Design reference	Code / file reference (planned)	Unit / integration TC	System / NFR / security TC	St.
FR-024	Staff / admin: correct or cancel records	4.5	C5, C6	AdminService.correctReservation · AuditLogger	admin/admin_service.cpp	IT-STF-02	SEC-T02	N
FR-025	Reports: bookings, occupancy, train-wise	4.5	C5	ReportService.* · IReportService	admin/report_service.cpp	IT-RPT-01, IT-RPT-02	ST-ADM-01	N
NFR-001	Search ≤ 2 s (P95)	5	C3, C7	Indexed queries (route_stop.station_code, schedule.journey_date) · §3.6	dal/sqlite_train_repo.cpp	–	PERF-01, PERF-02	N
NFR-002	Availability ≥ 99 %	5	C1–C8	Error handling §4.4 · Result<T> · ADR-003	main.cpp, ui/*	–	REL-01	N
NFR-003	Booking in ≤ 6 menu steps	5	C1	Menu trees and screens §4.5	ui/menu_*.cpp	–	ST-BKG-01, USB-01	N
NFR-004	No duplicate reservation on retry	5	C4, C7	request_id UNIQUE · ADR-004 · SD-01 steps 13-15	booking/booking_service.cpp	IT-BKG-06	REL-02	N
NFR-005	Modular, unit-tested business logic	5	All layers	Layered architecture · ADR-001 · module layout §4.1	src/*, tests/*	–	MNT-01	N
NFR-006	Helpers testable without I/O	5	C7	Repository interfaces · in-memory fakes · CD-02	booking/*, cancel/*, tests/fakes/*	–	TST-01	N
NFR-007	Portable GCC build	5	All	Technology stack §3.6 · no OS-specific API	CMakeLists.txt	–	PORT-01	N
SEC-001	No plaintext passwords	5.1	C6	PasswordHasher · §3.9.4	auth/password_hasher.cpp	UT-AUTH-02	SEC-T01	N
SEC-002	Role-based access	5.1	C6	AccessControl · RBAC matrix §3.9.3 · SD-03	security/access_control.cpp	IT-CAN-02	SEC-T02, SEC-T05, SEC-T08	N
SEC-003	Validate and sanitise input	5.1	C6, C7	InputValidator · bound SQL parameters · §3.9.5	security/input_validator.cpp, dal/*	UT-AUTH-01, UT-TRN-01, UT-BKG-02, UT-CAN-05	SEC-T03, SEC-T08	N
SEC-004	Reject invalid / expired session	5.1	C2, C6	Session.isValid · lock-out rule · SD-03	auth/session.cpp, auth/auth_service.cpp	UT-AUTH-04	ST-AUTH-02, SEC-T04, SEC-T06	N
SEC-005	Minimise retained ID data	5.1	C6, C7	PassengerProfile.maskedIdProof · retention job §3.9.6	security/privacy.cpp, dal/retention.cpp	–	SEC-T07	N
SEC-006	Log security events	5.1	C6, C8	AuditLogger · AuditEvent · §4.4	security/audit_logger.cpp	–	SEC-T05, SEC-T06	N

Appendix A – Field layouts

Layouts of the stored data (the Deliverable 2 fields plus the fields introduced by the design).

Field	Length	Data type	Description	Mandatory
passenger_id / account_id	36	UUID/String	Unique account (passenger) identifier	Y
role	10	Enum	PASSENGER, STAFF, ADMIN or OPS	Y
name	100	String	Passenger full name	Y
age	3	Integer	Passenger age (1–120)	Y
gender	1	Char	M, F or O	Y
contact_number	15	String	Mobile number, used for login/registration (unique)	Y
email	100	String	E-mail address (unique when present)	N
password_hash / salt	64 / 32	Hex string	PBKDF2-HMAC-SHA256 hash and random salt	Y
id_proof_type / id_proof_number	20 / 30	String	Identity-proof type and reference number	Y
train_id / train_number	36 / 10	UUID / String	Internal identifier / publicly displayed train number	Y
station_code / station_name	5 / 50	String	Station code and name (source, destination, stops)	Y
schedule_id / journey_date	36 / 10	UUID / Date	Journey-date instance of a train	Y
coach_class	20	Enum/String	Coach class (Sleeper, AC 3-Tier)	Y
seat_number	10	String	Allocated seat/berth number	N (until confirmed)
pnr_number	10	String	Unique reservation reference (10 digits)	Y
status	20	Enum/String	CONFIRMED / RAC / WAITLISTED / CANCELLED	Y
wl_position	3	Integer	Waitlist position (only for WAITLISTED)	N
fare_amount / refund_amount	10,2	Decimal	Calculated total fare / computed refund	Y / N
request_id	36	String	Idempotency token of the booking request (unique)	Y
event_time / event_type	19 / 30	DateTime / String	Audit event timestamp and type	Y

Report contents

Booking statistics report	Seat occupancy report	Train-wise reservation report
Date range Total reservations Count by status (Confirmed / RAC / Waitlisted / Cancelled) Total fare booked Total refunds	Train number and name Journey date and class Capacity, confirmed seats booked Occupancy % RAC and waitlist counts	Train number and name Number of reservations per train Number of travellers Breakdown by class Cancellation ratio

Appendix B – Project plan (from Deliverable 2)

Work breakdown

Work package	Tasks	Owner	Effort (person-days)	Output
WP1	Requirements, use cases, initial validation planning	All	5	SRS + initial validation spec
WP2	Architecture + UML design	Kshitij Prabhu / Catherine D	4	Architecture / design
WP3	Registration / login + passenger profile module	Bhadra R S	6	Working feature
WP4	Train search + schedule / route management module	Kshitij Prabhu	6	Working feature
WP5	Seat availability, booking, fare and PNR module	Catherine D	7	Working feature
WP6	Cancellation, waitlist and administrator module	K Pranav	7	Working feature
WP7	Testing + CI/CD + security validation	All	6	Test / validation evidence

Work package	Tasks	Owner	Effort (person-days)	Output
WP8	Final validation + consolidated report	All	4	Final report

Schedule (sprints 1-6)

Sprint	Planned output	Acceptance evidence
Sprint 1	Project setup, requirements baseline, user registration/login	User stories + Git repository + working login
Sprint 2	Train search, route and schedule management	Working search demo + unit tests
Sprint 3	Seat availability, allocation and fare calculation	Demo + unit/integration test evidence
Sprint 4	Reservation workflow, passenger details and PNR generation	End-to-end booking demo + PR evidence
Sprint 5	Cancellation, waitlist/status and administrator functions	Functional validation + coverage/security evidence
Sprint 6	System integration, validation, defect fixing and release	Final test report + working demonstration

Tools

C/C++17 with VS Code and GCC/G++; Git + GitHub; GitHub Projects (or Jira) for planning; Jenkins and Docker for CI/CD; GoogleTest with gcov/lcov; SonarQube and cppcheck for static analysis; Draw.io, StarUML and PlantUML for diagrams.

Part B – Software Architecture and Design Specification

Railway Reservation System – Train Scheduling, Seat Allocation and Ticket Management

Version	1.0
Authors	Team 14 – Bhadra R S, Kshitij Prabhu, Catherine D, K Pranav
Date	03-10-2026
Status	Draft 1.0 for evaluation – architecture and design baseline
Pattern	Layered architecture (Team 14, “Architecture Selection and Justification”)
Related documents	Part A – SRS v1.0 · Part C – Software Test Plan v1.0

Revision history

Version	Date	Author	Change summary
0.9	15-09-2026	Team 14	Layered architecture chosen; first component diagram.
1.0	03-10-2026	Team 14	Full SAD: component, class and deployment diagrams; decision records; STRIDE model and RBAC matrix; sequence, activity and state models; API design with data objects; error handling; UX; traceability.

Approvals

Role	Name	Signature / date
Course faculty	Dr. Sapna V M, PES University	
Team 14	Bhadra R S · Kshitij Prabhu · Catherine D · K Pranav	

1. Introduction

Purpose. This document describes how the Railway Reservation System (RRS) is built: the layered architecture and why it was chosen, each component and its interface, the UML models, the API of every component, error handling, security and the user interface. Every element is traced back to a requirement in the SRS (Part A). **Scope.** All five features (registration/login, train search, booking/fare/PNR, cancellation/waitlist, administration/reports) plus the shared security, audit and database services. Hardware and operating-system design are out of scope. **Audience.** The four developers, testers, security reviewers, faculty and future maintainers.

Term	Meaning
ADR	Architecture decision record – a decision, its alternatives and consequences.
DAO / repository	Object that hides the database behind an interface (IUserRepository and so on).
Result<T>	Return type holding either a value or an error code; used instead of exceptions across module boundaries.
STRIDE	Threat classes: Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege.
API / ABI	API = the C++ interface headers. ABI is not exposed: all modules are linked statically into one executable.
PNR, RAC, WL, RBAC	See SRS §1.4.

2. Document overview

Section 3 covers the architecture (goals, diagrams, components, pattern, technology, risks, traceability, security). Section 4 covers the design (modules and division of work, UML behaviour models, API, errors, UX, open issues). Figure IDs (CMP-01, CD-01, SD-01 ...) are the ones used in the SRS traceability matrix.

Figure	ID	Content	Section
B-1	CMP-01	Layered component diagram	3.3.1
B-2	CD-01	Domain class diagram	3.3.2
B-3	CD-02	Business-layer class diagram	3.3.2
B-4	DEP-01	Deployment diagram	3.3.3
B-5 – B-6	SD-01	Sequence – Book Ticket (a: fare quote, b: create reservation)	4.2.1
B-7 – B-8	SD-02	Sequence – Cancel Ticket (a: authorise and refund, b: apply and promote waitlist)	4.2.2
B-9 – B-10	SD-03	Sequence – Login (a) and access control (b)	4.2.3

Figure	ID	Content	Section
B-11	AD-01	Activity diagram – Book Ticket workflow	4.2.4
B-12	SM-01	State machine – ticket status	4.2.5

3. Architecture

3.1 Goals and constraints

Architectural driver	Goal / constraint	Source
Separation of concerns	UI, business rules and data access are independent, replaceable and individually testable	NFR-005, NFR-006
Role-based security	Passenger, staff, administrator and operations functions sit behind controlled business-layer interfaces	SEC-002, SEC-004
Performance	95 % of searches ≤ 2 s – direct in-process call path, indexed queries, no network hop	NFR-001
Reliability	No lost or duplicate reservation: ACID transactions, idempotent booking	NFR-002, NFR-004
Portability	Standard C++17 and SQLite only; no OS-specific API	NFR-007
Constraint – language	C and C++, modular object-oriented design	SRS §2.5
Constraint – deployment	Standalone console application; no network interface	SRS §3.4
Constraint – team	Four developers, each owning one functional area end to end	Project guidelines

3.2 Stakeholders and concerns

Stakeholder	Concerns	Addressed by
Passenger	Correct fare, seat and PNR; simple menus; privacy of ID data	Layered business rules (§3.5), UX (§4.5), SEC-005 (§3.9)
Reservation staff / administrator	Fast corrections, safe maintenance of trains, audit trail	Admin component C5, audit logging (§4.4)
Railway operations	Trustworthy reports	ReportService reads the same database through repositories
Developers	Clear module boundaries, stable interfaces, testability	Interfaces (§4.3), module ownership (§4.1)
Security reviewers	Threats mitigated, least privilege, audit	STRIDE, RBAC matrix (§3.9)
Course faculty	SDLC evidence: requirements → design → test traceability	Traceability (§3.8), SRS §8, STP §13

3.3 Component (UML) diagrams

3.3.1 Component diagram (CMP-01). The four business components match the four owned functional areas. They reach the database only through the *IDataAccess* interface. The three cross-cutting services – access control, input validation and audit logging – can be called by the UI and by every service. PasswordHasher is a helper used only by AuthService.

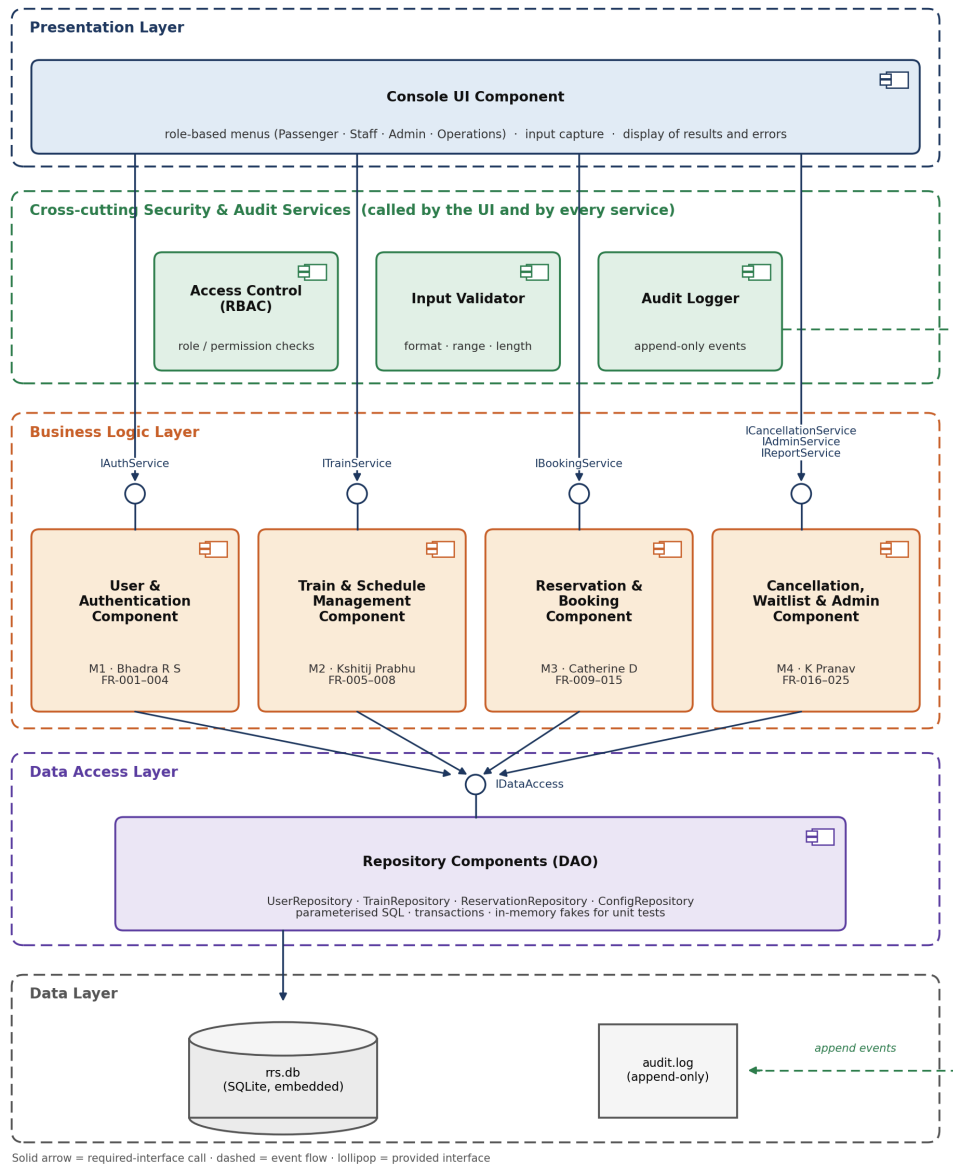


Figure B-1 – Component diagram of the layered architecture (CMP-01)

3.3.2 Class diagrams (CD-01, CD-02). CD-01 shows the persisted domain classes (see also the ER model, SRS Figure A-3). CD-02 shows the business-layer services, their helpers and the repository interfaces.

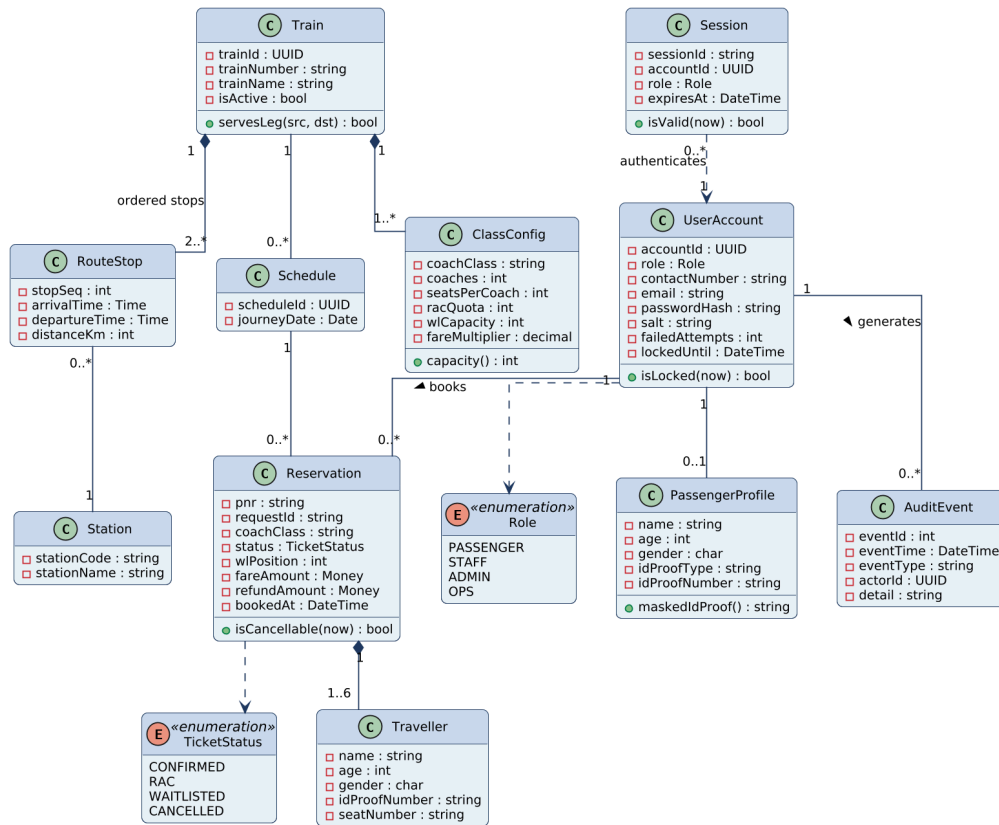
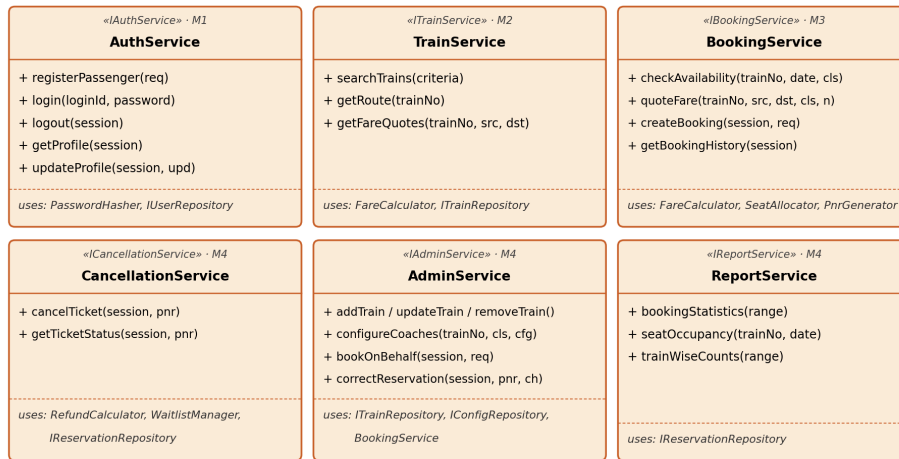
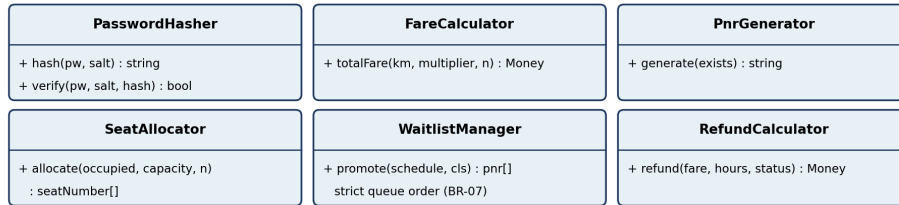


Figure B-2 – Domain class diagram (CD-01)

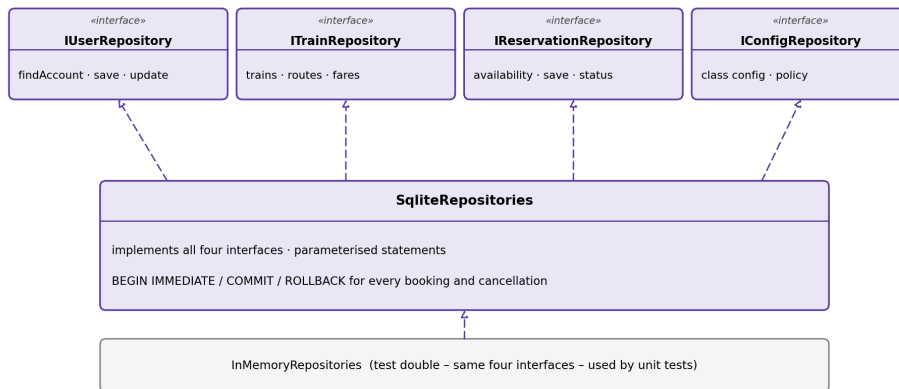
Business services (each realises the provided interface shown above its name)



Domain helpers - pure logic, no file or database I/O (testable in isolation, NFR-006)



Repository interfaces (Data Access Layer) - services depend on these, never on SQLite



«uses» = dependency listed inside each class · dashed arrow with hollow head = realisation (implements)

Figure B-3 – Business-layer class diagram (CD-02)

3.3.3 Deployment view (DEP-01). At run time RRS is one process on the user's machine with two local files. GitHub, Jenkins (in Docker) and the static-analysis tools build and test the same sources.

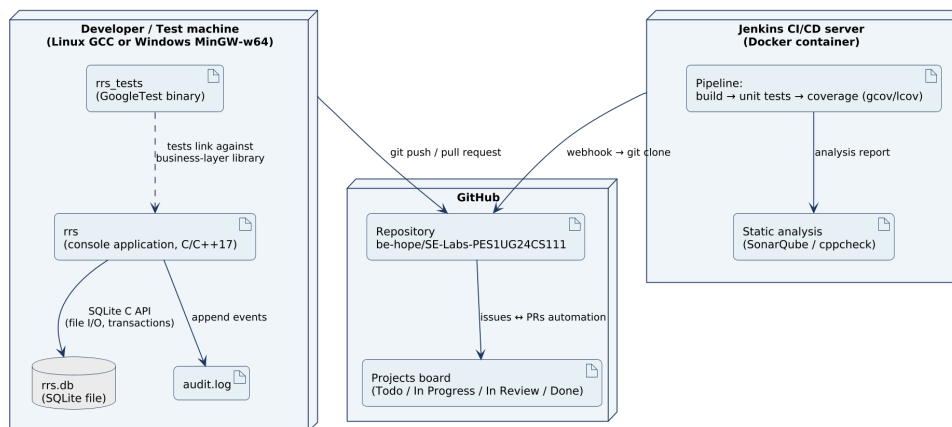


Figure B-4 – Deployment diagram (DEP-01)

3.4 Component descriptions

ID	Component (layer)	Responsibility	Provided interface(s)	Owner	Requirements
C1	Console UI (Presentation)	Role-based menus, input capture, display of results and errors; calls validators and services; no business rules	-	All	NFR-003, FR-003 (screens)
C2	User & Authentication (Business)	Registration, login, sessions, profile, lock-out	IAuthService	M1 Bhadra R S	FR-001 – 004, SEC-004
C3	Train & Schedule Management (Business)	Search, results, route, schedule, fare quotes	ITrainService	M2 Kshitij Prabhu	FR-005 – 008
C4	Reservation & Booking (Business)	Availability, traveller validation use, fare, seat allocation, PNR, status decision, history	IBookingService	M3 Catherine D	FR-009 – 015, NFR-004
C5	Cancellation, Waitlist & Administration (Business)	Cancellation, refund, waitlist promotion, status; train/coach admin; staff operations; reports	ICancellationService, IAdminService, IReportService	M4 K Pranav	FR-016 – 025
C6	Security & Audit Services (cross-cutting)	AccessControl (RBAC), InputValidator, AuditLogger; PasswordHasher helper used by C2	IAccessControl, IValidator, IAuditLogger	Shared; M1 lead	SEC-001 – 006
C7	Data Access (DAL)	Repositories with bound SQL, transactions, retention routine	IDataAccess (4 repository interfaces)	Shared; M3 lead	NFR-001, NFR-004, NFR-006
C8	Data stores	rrs.db (SQLite), audit.log (append-only)	-	-	SRS §5.2

3.5 Chosen architecture pattern and rationale

Pattern: layered architecture with four strict layers – Presentation → Business logic → Data access → Data – plus a cross-cutting security and audit group. A layer may call only the layer directly below it (and the cross-cutting services); the UI never calls a repository.

- **Separation of responsibilities:** user interaction, reservation rules, train management and persistence are separate modules (NFR-005).
- **Fits a standalone C/C++ program:** one process, one database file, no distributed-system overhead.
- **Security:** every privileged operation passes through a business-layer interface where RBAC is checked, so the UI cannot bypass it (SEC-002).
- **Performance:** a direct in-process call path keeps search and booking well inside the 2-second target (NFR-001).
- **Testability:** business logic depends on repository interfaces, so unit tests use in-memory fakes (NFR-006).

Alternatives considered

Alternative	Why it was not chosen
Client-server / microservices	Needs network, deployment and inter-service security that the standalone scope excludes; adds latency and complexity without benefit.
MVC with a monolithic controller	Controller tends to absorb business rules, making unit testing and ownership by four developers harder.
Single-file procedural C program	Fails the modularity and testability requirements; high coupling between menus, rules and file I/O.

Architecture decision records

ID	Decision	Context and rationale	Consequence
ADR-001	Layered architecture with strict downward calls	Team decision (Architecture Selection document); matches requirements and team ownership	Cross-layer shortcuts are forbidden and checked in review (MNT-01)
ADR-002	SQLite embedded database instead of flat files	Needs ACID transactions for seat allocation, UNIQUE constraints for accounts, PNR and request tokens; single file, no server; SRS allows 'local files or an embedded database'	Vendored SQLite amalgamation; repositories hide it, so files could replace it
ADR-003	Result<T> instead of exceptions across layer boundaries	Explicit, testable error paths with stable error codes (E-codes); no unwinding across module boundaries	Every call site checks ok(); helper macros reduce boilerplate
ADR-004	Idempotency token (request_id) on every booking	NFR-004: a retry after a lost confirmation must not create a second reservation	UNIQUE(request_id) in RESERVATION; BookingService looks the token up first
ADR-005	Strict-order waitlist promotion	BR-07: no skipping of the head of the queue	A large party at the head can block smaller tickets – accepted and documented

3.6 Technology stack and data stores

Area	Choice	Note
Language / standard	C++17 (C only for the vendored SQLite amalgamation)	Modular OO design
Compiler / build	GCC/G++ 11+, MinGW-w64; CMake (Make wrapper)	-Wall -Wextra -Werror; -fstack-protector-strong
Database	SQLite 3 (embedded), accessed with the C API and bound parameters	One file rrs.db; WAL off, synchronous = FULL
Audit log	Append-only text file audit.log	One line per event, no secrets
Cryptography	PBKDF2-HMAC-SHA256 (public-domain single-file implementation), OS random for salts	No external library
Unit testing / coverage	GoogleTest; gcov / lcov	Targets in NFR-005
Quality / security tools	cppcheck, clang-tidy, SonarQube, AddressSanitizer + UBSan, Valgrind	Run in CI
CI/CD, containers	Jenkins pipeline in a Docker container (gcc image)	Build → test → coverage → static analysis
Version control / board	Git + GitHub; GitHub Projects board (Todo / In Progress / In Review / Done) with PR-to-issue automation	Evidence of sprint activity

Data design – principal tables (ER model: SRS Figure A-3)

```

CREATE TABLE user_account (
  account_id TEXT PRIMARY KEY,
  role TEXT NOT NULL CHECK (role IN ('PASSENGER','STAFF','ADMIN','OPS')),
  contact_number TEXT NOT NULL UNIQUE, email TEXT UNIQUE,
  password_hash TEXT NOT NULL, salt TEXT NOT NULL,
  failed_attempts INTEGER NOT NULL DEFAULT 0, locked_until TEXT);
CREATE TABLE train (train_id TEXT PRIMARY KEY, train_number TEXT NOT NULL UNIQUE,
  train_name TEXT NOT NULL, is_active INTEGER NOT NULL);
CREATE TABLE route_stop (train_id TEXT NOT NULL REFERENCES train, stop_seq INTEGER NOT NULL,
  station_code TEXT NOT NULL, arrival_time TEXT, departure_time TEXT,
  distance_km INTEGER NOT NULL, PRIMARY KEY (train_id, stop_seq));
CREATE INDEX ix_route_station ON route_stop(station_code);
CREATE TABLE schedule (schedule_id TEXT PRIMARY KEY, train_id TEXT NOT NULL REFERENCES train,
  journey_date TEXT NOT NULL, UNIQUE (train_id, journey_date));
CREATE INDEX ix_schedule_date ON schedule(journey_date);
CREATE TABLE reservation (
  pnr_number TEXT PRIMARY KEY CHECK (length(pnr_number) = 10),
  schedule_id TEXT NOT NULL REFERENCES schedule,
  booked_by TEXT NOT NULL REFERENCES user_account,
  source_station TEXT NOT NULL, destination_station TEXT NOT NULL, coach_class TEXT NOT NULL,
  status TEXT NOT NULL CHECK (status IN ('CONFIRMED','RAC','WAITLISTED','CANCELLED')),
  wl_position INTEGER, fare_amount NUMERIC NOT NULL, refund_amount NUMERIC,
  request_id TEXT NOT NULL UNIQUE, booked_at TEXT NOT NULL);
CREATE TABLE traveller (traveller_id TEXT PRIMARY KEY,
  pnr_number TEXT NOT NULL REFERENCES reservation,
  schedule_id TEXT NOT NULL, coach_class TEXT NOT NULL, -- copied from the reservation
  name TEXT NOT NULL, age INTEGER NOT NULL CHECK (age BETWEEN 1 AND 120), gender TEXT NOT NULL,
  id_proof_number TEXT, seat_number TEXT); -- seat_number is NULL unless CONFIRMED
CREATE UNIQUE INDEX ux_seat ON traveller (schedule_id, coach_class, seat_number)
  WHERE seat_number IS NOT NULL; -- a seat can belong to one active ticket only (FR-013)

```

The partial unique index `ux_seat` makes double allocation impossible even if the application logic has a defect. The seat number is set to NULL when a ticket is cancelled, which releases the seat.

3.7 Risks and mitigations

Risk	Impact	Mitigation
Double allocation of a seat or PNR collision	High – two passengers, one seat	Allocation and insertion in one BEGIN IMMEDIATE transaction; UNIQUE constraints; PNR collision check with retry
Database file corruption on abrupt exit	High – lost bookings	SQLite journaling with synchronous = FULL; integrity check on start-up; backup copy before schema changes
Memory-safety defects in C/C++ code	High – crash or exploit	std::string / std::vector only, no gets/strcpy; ASan + UBSan and cppcheck in CI; input length limits
Interface drift between the four modules	Medium – integration failures	Headers in §4.3 frozen per sprint; contract tests; changes only by pull request with reviewer from the dependent module
SQLite build problems on Windows	Medium	Vendor the amalgamation (sqlite3.c) in the repository; build in CI on both platforms

Risk	Impact	Mitigation
Unequal contribution across members	Medium – evaluation	One end-to-end functional area each (\$4.1); per-member commits and PRs on the board
Schedule pressure before the final demo	Medium	Sprint plan with acceptance evidence (SRS Appendix C); High-priority requirements first

3.8 Traceability to requirements

Every requirement maps to at least one component and one design element. The per-requirement matrix with code files and test cases is SRS §8; this is the summary by group.

Requirements	Component(s)	Interface / design element
FR-001 – 004 registration, login, profile	C2, C6, C7	IAuthService; SD-03; UserAccount; PasswordHasher
FR-005 – 008 search, route, fare	C3, C4 (FareCalculator), C7	ITrainService; ITrainRepository
FR-009 – 015 availability, booking, PNR, history	C4, C6, C7	IBookingService; SD-01; AD-01; SeatAllocator; PnrGenerator
FR-016 – 020 cancel, refund, waitlist, status	C5, C7	ICancellationService; SD-02; SM-01; RefundCalculator; WaitlistManager
FR-021 – 025 admin, staff, reports	C5, C6, C7	IAdminService, IReportService; AuditLogger
NFR-001 – 007 quality attributes	All; C7 for NFR-001/004/006	Indexes, ADR-003/004, repository interfaces, build setup
SEC-001 – 006 security	C2, C6, C7, C8	§3.9 STRIDE, RBAC matrix, hashing, validation, audit log

3.9 Security architecture

3.9.1 Trust boundaries

Boundary	Crossing data	Control
User → Console UI	Untrusted keyboard input	InputValidator on every field before any service call (length, format, range)
UI → business services	Validated DTOs + Session	Each service entry point calls AccessControl.requirePermission()
Business → data access	Domain objects	Repository interfaces only; bound SQL parameters; no string-built SQL
Process → file system	rrs.db, audit.log	Files created with owner-only permissions (0600); audit.log opened append-only

3.9.2 Threat model (STRIDE)

Threat	STRIDE	Mitigation	Requirement
Guessing or replaying credentials	Spoofing	PBKDF2 hashes, generic login error, lock-out after 5 failures, 15-min session	SEC-001, SEC-004
Editing rrs.db or forging a ticket	Tampering	File permissions, UNIQUE/CHECK constraints, transactions, integrity check on start-up	SEC-003
Denying a cancellation or correction	Repudiation	Audit log with actor, time, old/new values for every privileged change	SEC-006
Reading passwords or ID numbers from storage, screen or logs	Information disclosure	Hashing; masked ID display; no secrets in logs; 90-day ID retention	SEC-001, SEC-005
Flooding the console with huge or malformed input; crashing the process	Denial of service	Length limits, validation, memory-safe containers, sanitizers in CI, lock-out throttling	SEC-003, NFR-002
Passenger reaching admin functions or other users' PNRs	Elevation of privilege	RBAC matrix, ownership check, server-side role from the session, never from user input	SEC-002

3.9.3 Role-based permission matrix

Y = permitted, Y (own) = own records only, – = denied (ERR-FORBIDDEN, E1006). The matrix is the single source for AccessControl and for test SEC-T02.

Operation (permission)	Anonymous	Passenger	Staff	Admin	Ops
Register, login	Y	–	–	–	–
Search trains, route, fare, availability	Y	Y	Y	Y	Y
Book ticket (BOOK_TICKET)	–	Y (own)	Y (on behalf)	–	–
Profile, booking history	–	Y (own)	–	–	–
PNR status (VIEW_PNR)	–	Y (own)	Y	Y	–
Cancel ticket (CANCEL_TICKET)	–	Y (own)	Y	Y	–

Operation (permission)	Anonymous	Passenger	Staff	Admin	Ops
Correct / override reservation (EDIT_RESERVATION)	-	-	Y	Y	-
Manage trains, routes, schedules, coaches (MANAGE_TRAINS)	-	-	-	Y	-
Reports (VIEW_REPORTS)	-	-	-	Y	Y
View audit log (VIEW_AUDIT)	-	-	-	Y	-

3.9.4 Authentication and session handling

- Passwords are hashed with PBKDF2-HMAC-SHA256 using a 16-byte random salt per account and at least 10,000 iterations (a higher count such as 600,000, as OWASP now recommends, only needs a configuration change); only the hash and salt are stored (SEC-001).
- login() takes the same time and returns the same message for an unknown id and a wrong password; an unknown id still runs a dummy verification so timing does not reveal which accounts exist.
- A Session holds account id, role and expiry (now + 15 minutes, extended on each operation). The role always comes from the session created at login, never from user input (SEC-004).
- Five consecutive failed logins set locked_until = now + 5 minutes; a success resets the counter; both events are audited.

3.9.5 Input validation and injection defence

- InputValidator has one function per field type (mobile, e-mail, password, name, age, gender, ID proof, station code, date, PNR) and returns Result<T>. Free text is limited to 100 printable characters.
- All SQL uses prepared statements with bound parameters; no SQL text is built from user input (SEC-003).
- No fixed-size C buffers: input is read with std::getline into std::string and length-checked.

3.9.6 Data protection, audit and secure build

- ID-proof numbers are shown masked (last 4 characters), never logged, and cleared 90 days after the journey (SEC-005).
- AuditLogger appends lines such as 2026-10-03T14:35:22 | LOGIN_FAILED | actor=9876500001 | detail=bad password. Passwords and ID numbers never appear (SEC-006).
- Build flags: -Wall -Wextra -Werror -fstack-protector-strong -D_FORTIFY_SOURCE=2; CI runs AddressSanitizer/UBSan, cppcheck and SonarQube.

4. Design

4.1 Design overview and division of work

There is one module per business component plus the shared security, data-access and UI modules. Each member owns one functional area end to end (screens, service, tests) and reviews the pull requests of a second member. Shared modules have a lead but change only by pull request.

Member	Owned functionality	Module	Main classes	Depends on	Peer reviewer
Bhadra R S (M1)	Registration, login, profile	auth/	AuthService, Session, PasswordHasher	security/ (InputValidator, AuditLogger); dal/ (IUserRepository)	Catherine D
Kshitij Prabhu (M2)	Train search, schedule, route, fare display	train/	TrainService, Train.servesLeg	dal/ (ITrainRepository); booking/fare_calculator (M3); security/ (InputValidator)	K Pranav
Catherine D (M3)	Availability, booking, fare, PNR, history	booking/	BookingService, FareCalculator, SeatAllocator, PnrGenerator	dal/ (IReservationRepository, ITrainRepository, IUnitOfWork); security/ (AccessControl, InputValidator, AuditLogger)	Bhadra R S
K Pranav (M4)	Cancellation, refund, waitlist, admin, staff, reports	cancel/, admin/	CancellationService, RefundCalculator, WaitlistManager, AdminService, ReportService	booking/ (BookingService for book-on-behalf, SeatAllocator); dal/ (IReservationRepository, ITrainRepository, IConfigRepository); security/	Kshitij Prabhu
Shared (lead M1)	Access control, validation, audit, hashing	security/	AccessControl, InputValidator, AuditLogger, PasswordHasher	common/ only (audit.log file)	-
Shared (lead M3)	Repositories, transactions, retention	dal/	Sqlite repositories, IUnitOfWork, retention routine	common/, third_party/ (SQLite)	-
All	Console menus per role	ui/	menu_*.cpp	every service interface; security/	-

Planned source tree

```
src/
  main.cpp          application start-up, wiring of components
  ui/               menu_*.cpp      C1 console menus per role
  auth/             auth_service.* session.* password_hasher.*      C2 (M1)
  train/            train_service.*      C3 (M2)
  booking/          booking_service.* fare_calculator.* seat_allocator.* pnr_generator.*      C4 (M3)
  cancel/           cancellation_service.* refund_calculator.* waitlist_manager.*      C5 (M4)
  admin/            admin_service.* report_service.*      C5 (M4)
  security/         access_control.* input_validator.* audit_logger.* privacy.*      C6
  dal/              i_*_repository.h sqlite_*_repo.* retention.cpp      C7
  common/           result.h error_codes.h money.h types.h
  tests/            unit/ integration/ system/ fakes/               GoogleTest
  third_party/      sqlite3.c sqlite3.h pbkdf2.*                   vendored
  CMakeLists.txt Jenkinsfile Dockerfile README.md
```

Coding conventions

- Headers expose interfaces only; implementation stays in .cpp files.
- Business classes get their repositories through the constructor (testability, NFR-006).
- Money is a fixed-point type (paise as integer); no floating point in fare or refund arithmetic.
- Every service operation returns Result<T>; no exception crosses a module boundary (ADR-003). Small pure helpers (FareCalculator, RefundCalculator, PasswordHasher) return plain values.

4.2 UML sequence diagrams and behavioural models

Three flows are modelled: booking a ticket (SD-01), cancelling with waitlist promotion (SD-02), and login with access control (SD-03). Each is split over two figures to stay readable. Solid arrows are calls, dashed arrows are returns, frames are alternatives.

4.2.1 SD-01 – Book Ticket (FR-009 to FR-014, FR-018, NFR-004)

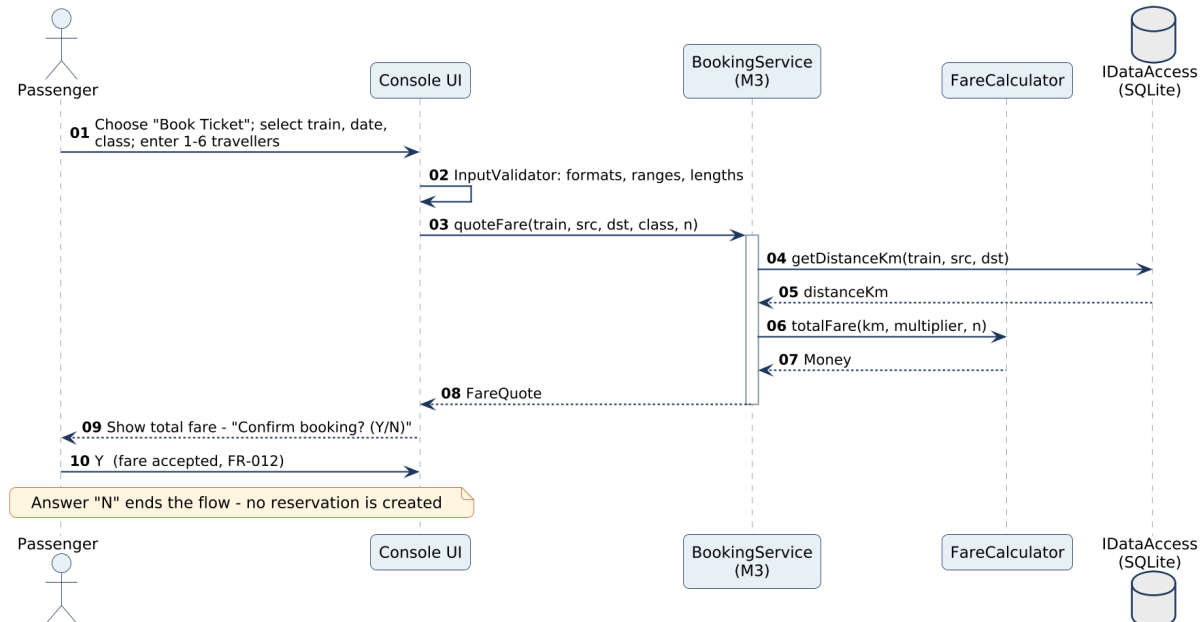


Figure B-5 – SD-01a: fare quote and acceptance

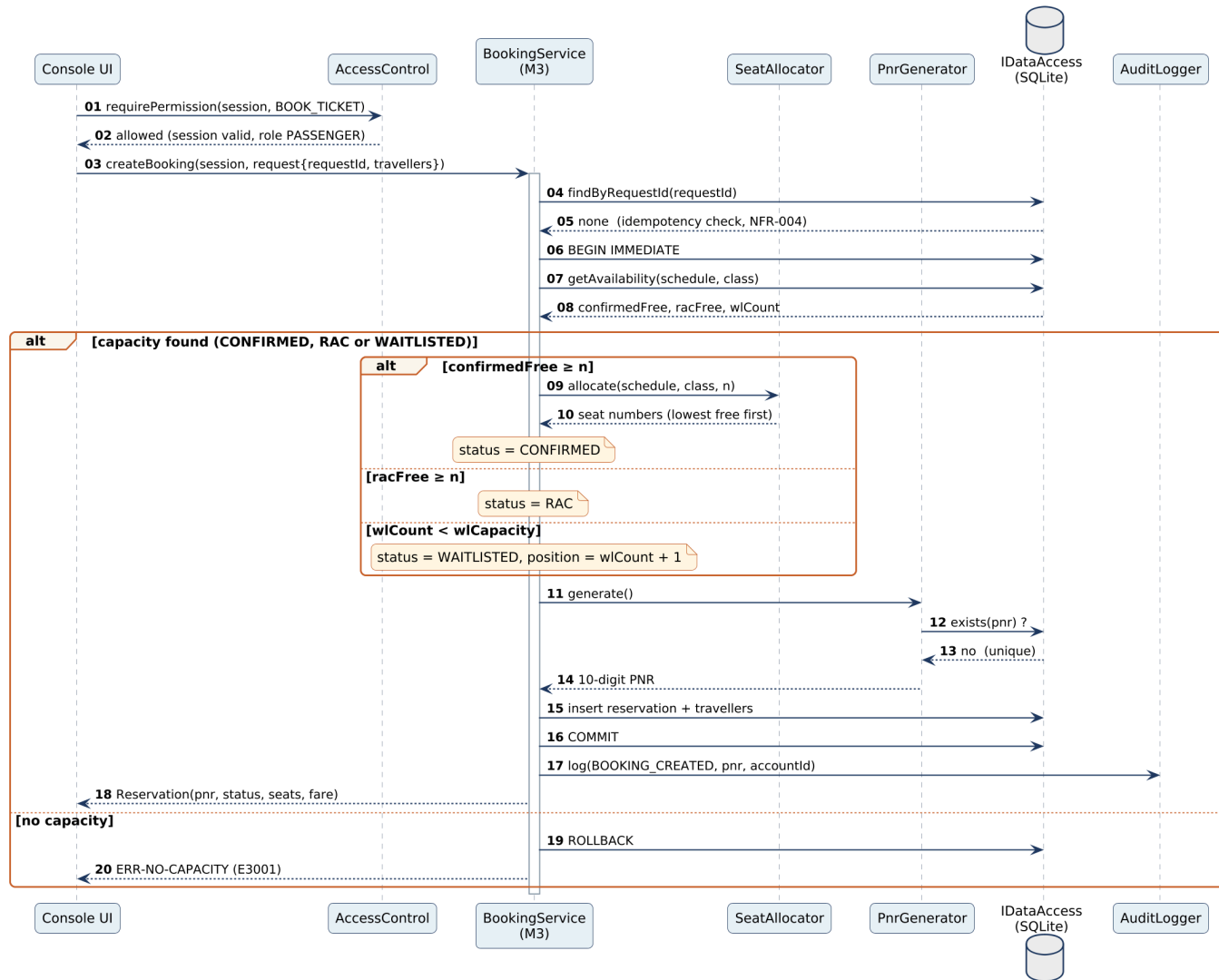


Figure B-6 – SD-01b: creating the reservation in one transaction (status decision, seat allocation, PNR, audit)

4.2.2 SD-02 - Cancel Ticket (FR-016 to FR-020)

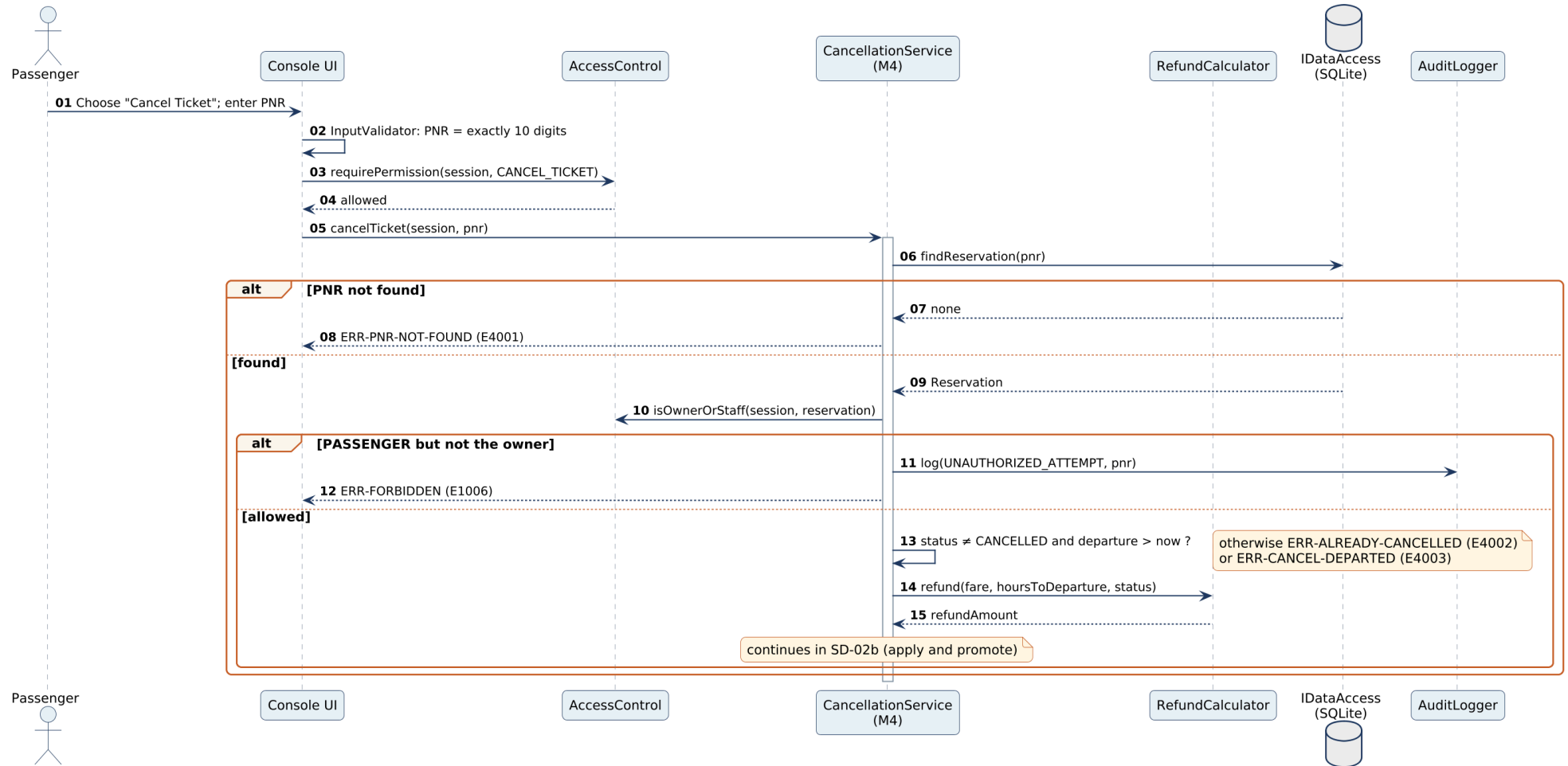


Figure B-7 – SD-02a: validation, authorisation, eligibility and refund

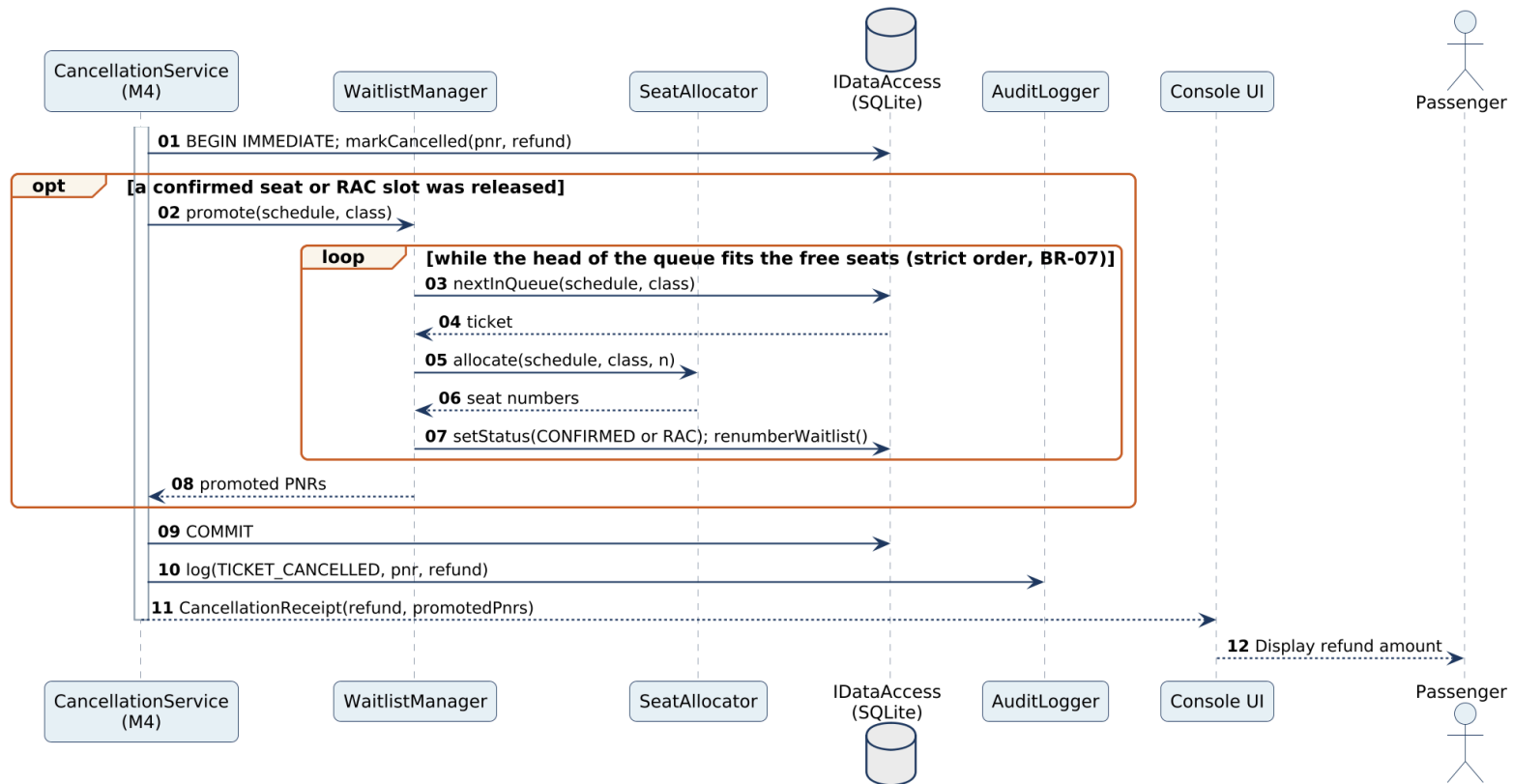


Figure B-8 – SD-02b: applying the cancellation and promoting the waitlist in strict order

4.2.3 SD-03 – Login and access control (FR-002, SEC-002, SEC-004, SEC-006)

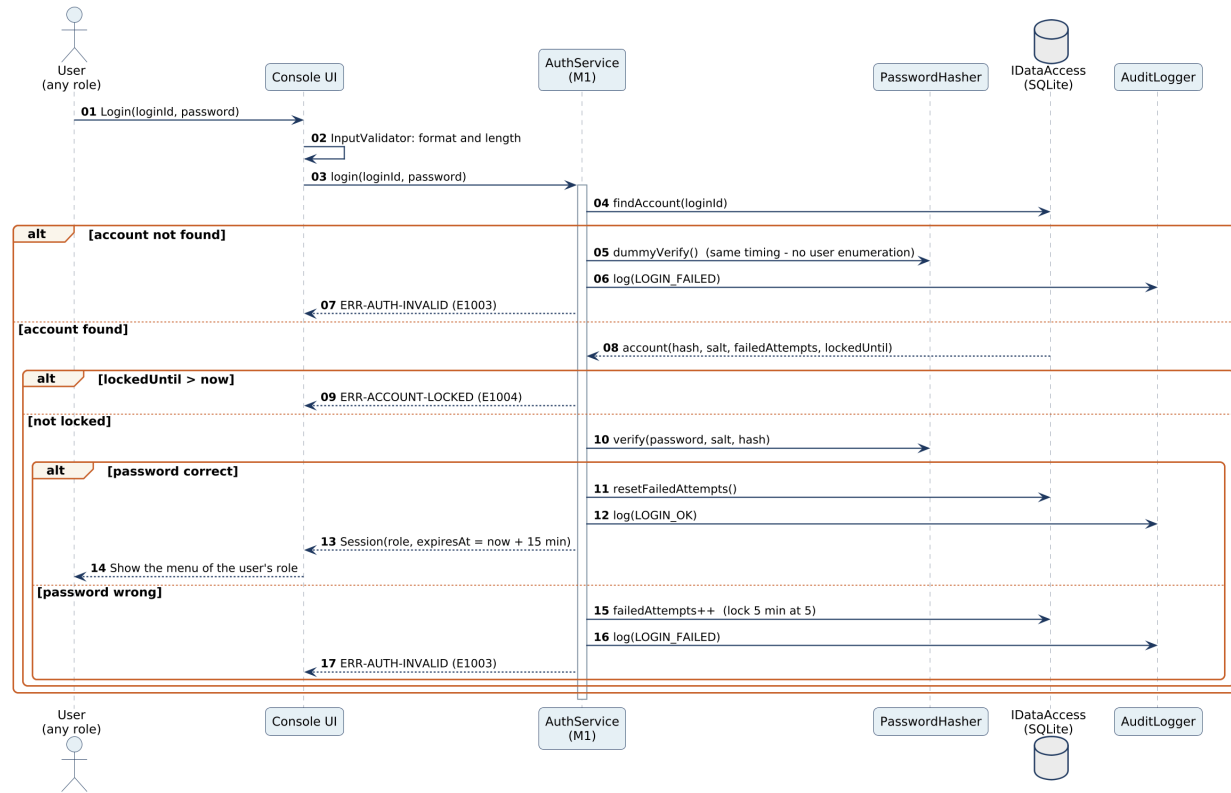


Figure B-9 – SD-03a: login with lock-out and generic errors

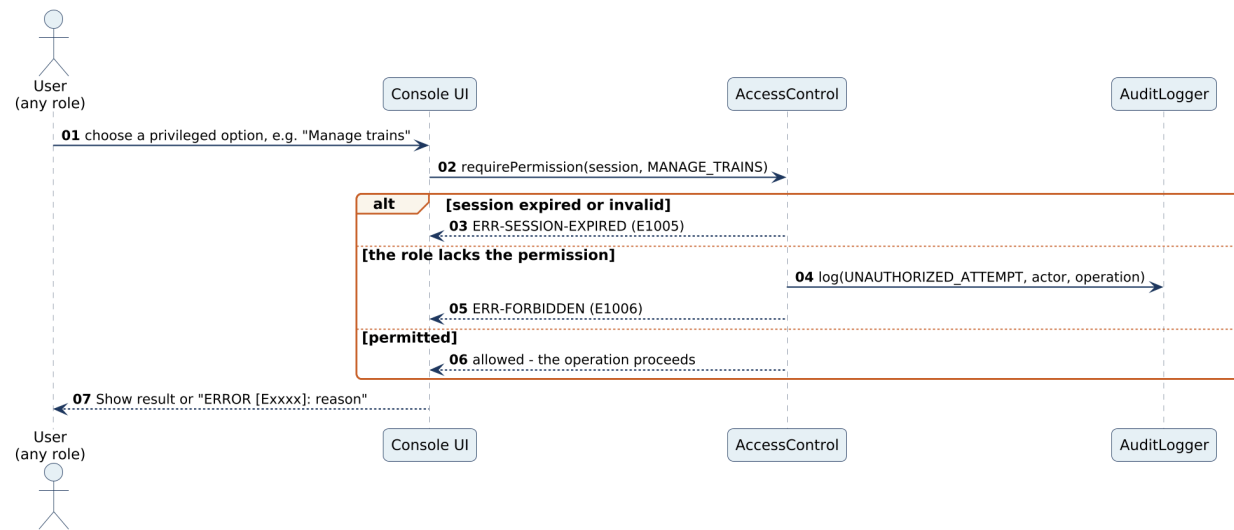


Figure B-10 – SD-03b: permission check before every privileged operation

4.2.4 AD-01 – Book Ticket workflow (activity diagram)

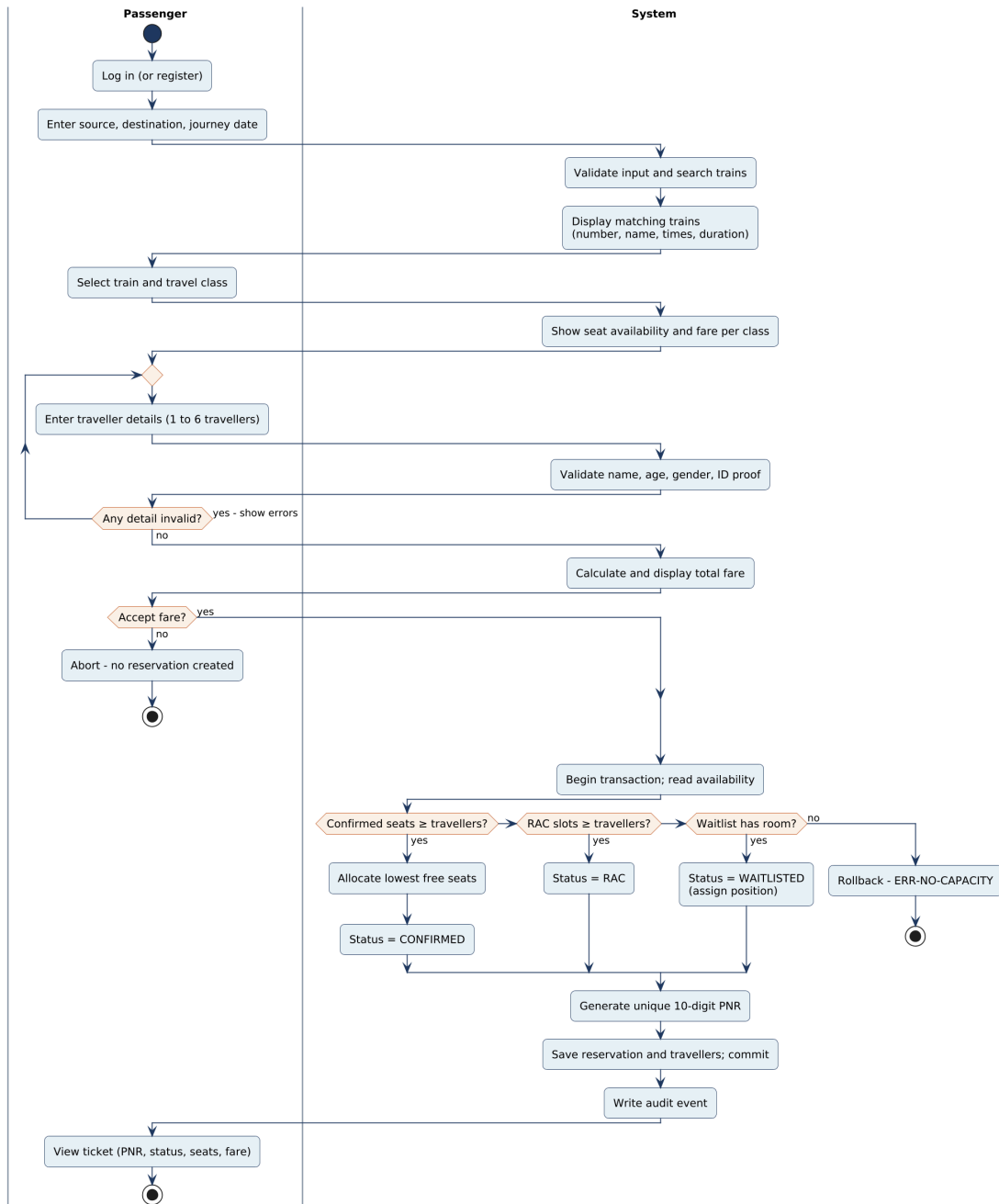


Figure B-11 – Activity diagram of the booking workflow (AD-01)

4.2.5 SM-01 – Ticket status (state machine)

A ticket starts as CONFIRMED, RAC or WAITLISTED and ends as CANCELLED. Promotion happens only inside the cancellation transaction, in strict queue order (BR-07, ADR-005).

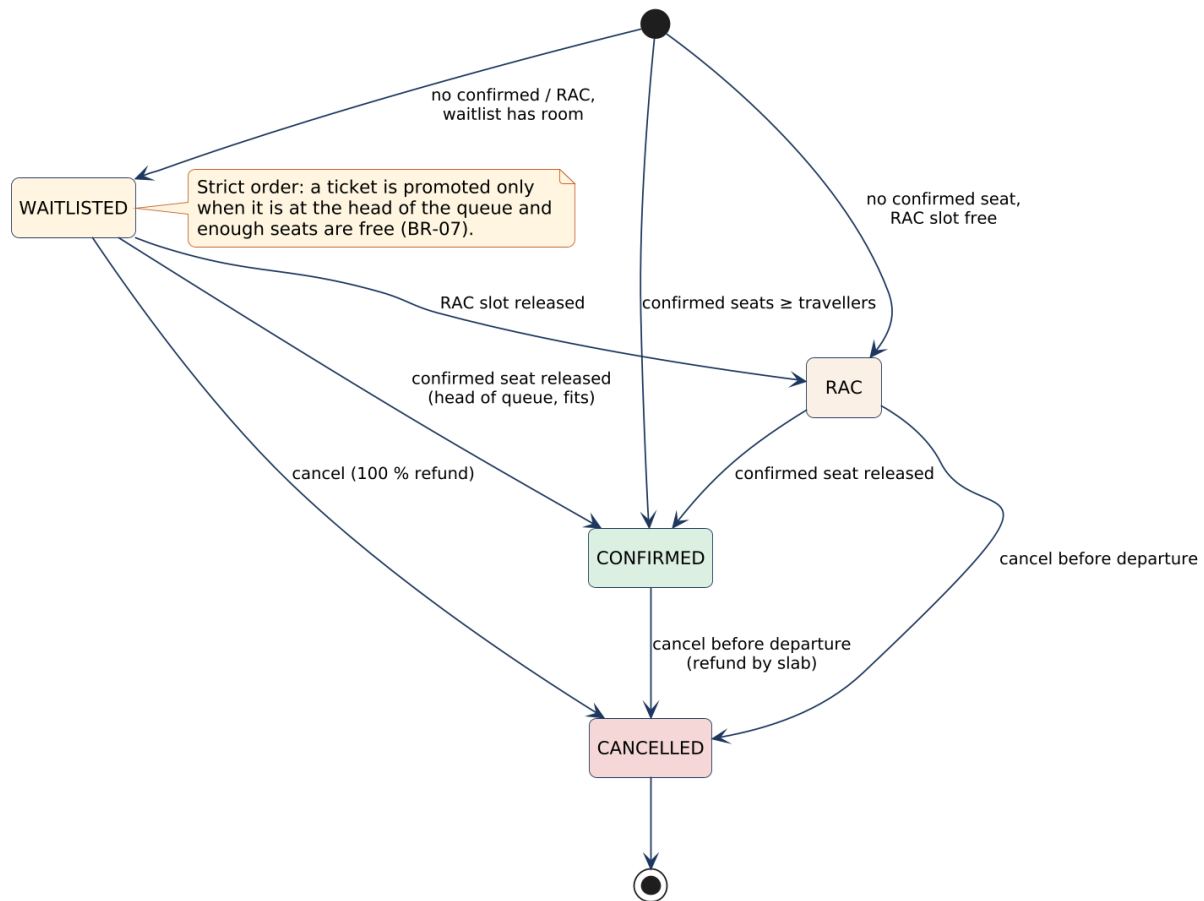


Figure B-12 – State machine of a ticket (SM-01)

4.3 API design

Each component exposes a C++ abstract interface in its header. The tables list the operations with the required role, the result and the error codes (catalogue in §4.4). All operations return `Result<T>`; the `Session` is passed explicitly and checked by `AccessControl`.

```

// common/result.h
enum class Err {
    Ok = 0,
    Validation = 1001, DupUser = 1002, AuthInvalid = 1003, AccountLocked = 1004,
    SessionExpired = 1005, Forbidden = 1006,
    TrainNotFound = 2001, NoCapacity = 3001,
    PnrNotFound = 4001, AlreadyCancelled = 4002, CancelDeparted = 4003,
    TrainInUse = 5001, CapacityBelowBooked = 5002, Db = 9001, Internal = 9999 };
template <class T> struct Result { bool ok; T value; Err error; std::string message; };

// booking/i_booking_service.h (M3) - an example of an interface header
class IBookingService {
public:
    virtual ~IBookingService() = default;
    virtual Result<Availability> checkAvailability(const std::string& trainNo,
                                                const Date& date, ClassCode cls) = 0;
    virtual Result<Money> quoteFare(const std::string& trainNo, const std::string& src,
                                   const std::string& dst, ClassCode cls, int travellers) = 0;
    virtual Result<Reservation> createBooking(const Session& s, const BookingRequest& req) = 0;
    virtual Result<std::vector<Reservation>> getBookingHistory(const Session& s) = 0;
};
  
```

IAuthService (C2 – M1)

Operation	Role / precondition	Returns	Errors
registerPassenger(RegistrationRequest)	Anonymous	Result<AccountId>	E1001, E1002
login(loginId, password)	Anonymous	Result<Session>	E1001, E1003, E1004
logout(Session)	Any session	Result<void>	E1005

Operation	Role / precondition	Returns	Errors
getProfile(Session) / updateProfile(Session, ProfileUpdate)	PASSENGER	Result<PassengerProfile> / Result<void>	E1001, E1005, E1006

ITrainService (C3 – M2)

Operation	Role / precondition	Returns	Errors
searchTrains(SearchCriteria{src, dst, date})	Anonymous	Result<vector<TrainSummary>>	E1001
getRoute(trainNumber)	Anonymous	Result<vector<RouteStop>>	E2001
getFareQuotes(trainNumber, src, dst)	Anonymous	Result<vector<FareQuote>>	E1001, E2001

IBookingService (C4 – M3)

Operation	Role / precondition	Returns	Errors
checkAvailability(trainNo, date, cls)	Anonymous	Result<Availability>	E1001, E2001
quoteFare(trainNo, src, dst, cls, n)	Anonymous	Result<Money>	E1001, E2001
createBooking(Session, BookingRequest)	PASSENGER or STAFF; valid session	Result<Reservation>	E1001, E1005, E1006, E3001
getBookingHistory(Session)	PASSENGER (own)	Result<vector<Reservation>>	E1005

ICancellationService (C5 – M4)

Operation	Role / precondition	Returns	Errors
cancelTicket(Session, pnr)	Owner, STAFF, ADMIN	Result<CancellationReceipt{refund, promotedPnrs}>	E1006, E4001, E4002, E4003
getTicketStatus(Session, pnr)	Owner, STAFF, ADMIN	Result<TicketStatusInfo>	E1006, E4001

IAdminService and IReportService (C5 – M4)

Operation	Role / precondition	Returns	Errors
addTrain / updateTrain(Session, TrainSpec); removeTrain(Session, trainNo)	ADMIN	Result<void>	E1001, E1006, E5001
configureCoaches(Session, trainNo, cls, ClassConfig)	ADMIN	Result<void>	E1001, E5002
bookOnBehalf(Session, BookingRequest)	STAFF	Result<Reservation>	as createBooking
correctReservation(Session, pnr, Correction)	STAFF, ADMIN	Result<void>	E1001, E1006, E4001
bookingStatistics(Session, DateRange); seatOccupancy(Session, trainNo, date); trainWiseCounts(Session, DateRange)	ADMIN, OPS	Result<Report>	E1001, E1006

Security and audit services (C6)

Operation	Role / precondition	Returns	Errors
AccessControl::requirePermission(Session, Permission)	Called by every service	Result<void>	E1005, E1006
AccessControl::isOwnerOrStaff(Session, Reservation)	Cancellation, status	bool	-
InputValidator::validateMobile / Email / Password / Traveller / Date / Pnr(...)	UI and services	Result<T>	E1001
AuditLogger::log(EventType, actorId, detail)	All services	void (never throws)	-

IDataAccess – repository interfaces (C7)

Operation	Role / precondition	Returns	Errors
IUserRepository: findAccount(loginId), insert(account), updateAttempts(id, n, lockedUntil)	Business layer	Result<Account>	E1002, E9001
ITrainRepository: findTrains(src, dst, date), getRoute(no), distanceKm(no, src, dst), save(Train)	Business layer	Result<...>	E2001, E9001
IReservationRepository: getAvailability, findByPnr, findByRequestId, insert, setStatus, nextInQueue, renumberWaitlist, listForReport	Business layer	Result<...>	E4001, E9001
IUnitOfWork: begin(), commit(), rollback() (BEGIN IMMEDIATE)	Booking, cancellation	Result<void>	E9001

Data objects (request and response schemas)

Field rules repeat the SRS limits; all of them are enforced once, in InputValidator, before a service is called. “Out” fields are filled by the system.

Object	Fields and validation rules	Used by
RegistrationRequest	loginId – 10-digit mobile or e-mail (≤ 100 chars, valid format, unique); name – 1–100 chars; password – ≥ 8 chars with at least one letter and one digit	registerPassenger
Session (out)	sessionId – string; accountId – UUID; role – PASSENGER STAFF ADMIN OPS; expiresAt – now + 15 min, refreshed on each operation	login, every service
ProfileUpdate	name 1–100; age 1–120; gender M F O; contactNumber ≤ 15 digits; idProofType ≤ 20 ; idProofNumber ≤ 30 (all mandatory)	updateProfile
SearchCriteria	sourceCode, destinationCode – station code ≤ 5 chars, must exist and differ; journeyDate – DD-MM-YYYY, today to today + 120 days	searchTrains
TrainSummary (out)	trainNumber ≤ 10 ; trainName ≤ 60 ; departure and arrival (HH:MM); duration (hh:mm, correct across midnight)	searchTrains
RouteStop (out)	stopSeq – int; stationCode; arrival, departure – HH:MM; distanceKm – cumulative km	getRoute
FareQuote (out)	classCode – SL 3A; fare – decimal(10,2) = km $\times$ ₹1.00 $\times$ class multiplier	getFareQuotes, quoteFare
Availability (out)	confirmedFree, racFree, waitlistCount, waitlistCapacity – non-negative integers	checkAvailability
BookingRequest	requestId – unique string ≤ 36 , new per booking, reused on retry; trainNumber; journeyDate; source; destination; classCode; travellers – list of 1–6 Traveller	createBooking, bookOnBehalf
Traveller	name 1–100; age 1–120; gender M F O; idProofNumber ≤ 30 (mandatory); seatNumber (out) – NULL unless CONFIRMED	BookingRequest, Reservation
Reservation (out)	pnr – 10 digits, first digit 1–9; status – CONFIRMED RAC WAITLISTED CANCELLED; wlPosition – only for WAITLISTED; fareAmount – decimal(10,2); travellers with seats; bookedBy; bookedAt	createBooking, getBookingHistory
CancellationReceipt (out)	pnr; refundAmount – decimal(10,2); promotedPnrs – list of PNRs promoted in the same transaction	cancelTicket
TicketStatusInfo (out)	pnr; status; wlPosition (only if WAITLISTED)	getTicketStatus
TrainSpec	trainNumber ≤ 10 , unique; trainName ≤ 60 ; stops – at least 2, unique stations, strictly increasing times and distanceKm; isActive	addTrain, updateTrain
ClassConfig	classCode; coaches ≥ 1 ; seatsPerCoach ≥ 1 ; racQuota ≥ 0 ; wlCapacity ≥ 0 ; fareMultiplier > 0 ; capacity may not drop below the seats already booked	configureCoaches
Correction	pnr; changes to traveller name, age, gender or ID proof, or a cancel flag; every change is audited with actor, time, old and new value	correctReservation
DateRange	from, to – DD-MM-YYYY, from $\leq$ to	bookingStatistics, trainWiseCounts
Report (out)	rows defined in SRS Appendix A (booking statistics, seat occupancy, train-wise counts)	ReportService

API contract rules

- The interface headers are the contract between modules. A change needs a pull request approved by the owner of every dependent module and a bump of the interface version comment (current version: 1).
- A signature, field rule or error code is never changed silently after another module depends on it.
- All modules link statically into one executable, so binary compatibility is guaranteed by rebuilding; no plug-in or shared-library ABI is exposed.
- Field validation is implemented once, in InputValidator.

4.4 Error handling, logging and monitoring

Errors are values (Result<T>), not exceptions. A failed validation never reaches the database; a database failure rolls the transaction back and returns E9001; the UI turns an error code into one standard line: `ERROR [E3001]: No seats or waitlist room on this train. Choose another date or class.`

Code	Name	Raised by	Message shown to the user	Log / audit
E1001	ERR-VALIDATION	InputValidator	Invalid <field>; <reason>. Please re-enter.	none
E1002	ERR-DUP-USER	AuthService	An account with this mobile number or e-mail already exists.	info
E1003	ERR-AUTH-INVALID	AuthService	Invalid login ID or password.	audit LOGIN_FAILED

Code	Name	Raised by	Message shown to the user	Log / audit
E1004	ERR-ACCOUNT-LOCKED	AuthService	Too many failed attempts. Try again in 5 minutes.	audit ACCOUNT_LOCKED
E1005	ERR-SESSION-EXPIRED	AccessControl	Your session has expired. Please log in again.	audit
E1006	ERR-FORBIDDEN	AccessControl, services	You are not allowed to perform this operation.	audit UNAUTHORIZED_ATTEMPT
E2001	ERR-TRAIN-NOT-FOUND	Train / booking	Train not found.	none
E3001	ERR-NO-CAPACITY	BookingService	No seats or waitlist room on this train.	info
E4001	ERR-PNR-NOT-FOUND	Cancellation	PNR not found.	none
E4002	ERR-ALREADY-CANCELLED	Cancellation	This ticket is already cancelled.	none
E4003	ERR-CANCEL-DEPARTED	Cancellation	The train has already departed; the ticket cannot be cancelled.	none
E5001	ERR-TRAIN-IN-USE	AdminService	The train has active reservations and cannot be removed.	audit
E5002	ERR-CAPACITY-BELOW-BOOK ED	AdminService	Capacity cannot be lower than the seats already booked.	audit
E9001	ERR-DB	Repositories	The data could not be saved. Changes were not applied.	error
E9999	ERR-INTERNAL	Any	An unexpected error occurred. Please try again.	error

- **audit.log** (append-only): LOGIN_OK, LOGIN_FAILED, ACCOUNT_LOCKED, UNAUTHORIZED_ATTEMPT, BOOKING_CREATED, TICKET_CANCELLED, RESERVATION_CORRECTED, TRAIN_CHANGED. No passwords or ID numbers.
- **Application log** (rrs.log): INFO / WARN / ERROR with timestamp and component, same no-secrets rule.
- **Metrics** read from the logs and database by the soak test and the operations report: search latency P50/P95, booking failure rate, E9001 count, waitlist length per train, failed logins per hour.
- **Start-up checks**: SQLite integrity check; if the data files are missing, a new database is created and the user is told.

4.5 UX design

A numbered console menu per role. A new passenger can finish a booking in six primary steps (NFR-003): Login → Search → Select train and class → Enter travellers → Confirm fare → View ticket.

Role	Menu entries	Requirements
Anonymous	1 Login · 2 Register · 3 Search trains · 4 Schedule, route & fare · 0 Exit	FR-001, FR-002, FR-005, FR-007
Passenger	1 Search trains · 2 Schedule, route & fare · 3 Check availability · 4 Book ticket · 5 My bookings · 6 PNR status · 7 Cancel ticket · 8 My profile · 0 Logout	FR-003, FR-005–FR-020
Reservation staff	1 Search trains · 2 Book on behalf · 3 PNR status · 4 Cancel / correct reservation · 0 Logout	FR-005, FR-020, FR-023, FR-024
System administrator	1 Trains, routes & schedules · 2 Coaches & seats · 3 Reservation records · 4 Reports · 5 Audit log · 0 Logout	FR-021, FR-022, FR-024, FR-025, SEC-006
Railway operations	1 Booking statistics · 2 Seat occupancy · 3 Train-wise counts · 4 Schedules · 0 Logout	FR-025, FR-007

SEARCH RESULTS

SBC -> MAS

journey date 10-10-2026

Train Name

Dep

Arr

Duration

SL / 3A seats

1 20001 Bengaluru-Chennai Express

06:00

11:30

05:30

212 / 96

Select train number (0 = back) > 20001

FARE AND CONFIRMATION

Train 20001 SBC -> MAS

Class : AC 3-Tier

Travellers : 2

Distance : 350 km

Fare : 350 km x 2.5 x 2

TOTAL : Rs 1,750.00

Confirm booking? (Y/N) > Y

OK: Ticket confirmed.

PNR 4829163057

Status CONFIRMED

Seats S1-07, S1-08

(Asha Rao, 28 F | ID *****0001)

ERROR [E4003]: The train has already departed; the ticket cannot be cancelled.

ERROR [E1001]: Invalid age: enter a number from 1 to 120. Please re-enter.

Every screen shows the user's role; option 0 always goes back; prompts name the expected format; errors state the reason and the action; passwords are never echoed. Colour is not needed to read a screen, and lines fit in 80 columns.

4.6 Open issues, next steps and design-to-validation update

Open issue / next step	Planned in
Implement the interface headers of §4.3 and freeze them for Sprint 2	Sprint 1–2
Vendor SQLite and PBKDF2, create the CMake / Docker / Jenkins pipeline and the GitHub Projects board	Sprint 1–2
Partial (per-passenger) cancellation of a multi-traveller PNR – out of the current scope	Future
Concurrent access from several terminals (would need a server and a new thread model)	Future
Payment-gateway integration (mock adapter first)	Future

Design-to-validation

Design element	Validation question	Planned test
RBAC matrix (§3.9.3)	Can a passenger call a staff or administrator function?	SEC-T02, SEC-T08
Ticket state machine (SM-01)	Can an invalid transition occur; is promotion strictly ordered?	UT-CAN-03, UT-CAN-04, IT-CAN-03, IT-CAN-05
Idempotent booking (ADR-004)	Does a retry create a second reservation?	IT-BKG-06, REL-02
Repository interfaces (CD-02)	Does the business logic work without a database?	TST-01, UT-BKG-0x
Database constraints (§3.6)	Can duplicate accounts, PNRs or seats be inserted?	IT-AUTH-02, UT-BKG-03, IT-BKG-05
Session and lock-out rules (§3.9.4)	Do sessions expire and accounts lock?	ST-AUTH-02, SEC-T04, SEC-T06
Input validation (§3.9.5)	Is hostile input rejected cleanly?	UT-AUTH-01, SEC-T03

5. References

- IEEE 42010:2011 – Architecture description. OWASP Top 10 (2021), OWASP ASVS, MITRE CWE, Microsoft STRIDE method.
- NIST SP 800-160 vol. 1; NIST SP 800-132 (password-based key derivation). SQLite documentation (transactions, prepared statements).
- Team 14: Architecture Selection and Justification; Deliverable 2 – Requirements (2026).
- Diagrams drawn with PlantUML, Graphviz and Python/matplotlib.

Part C – Software Test Plan

Railway Reservation System – Train Scheduling, Seat Allocation and Ticket Management

Version	1.0
Authors	Team 14 – Bhadra R S, Kshitij Prabhu, Catherine D, K Pranav
Date	03-10-2026
Status	Draft 1.0 for evaluation – 72 designed test cases, none executed yet
Standard	IEEE Std 829-2008 structure as given in the course template (15 sections)
Related documents	Part A – SRS v1.0 · Part B – SAD v1.0

Revision history

Version	Date	Author	Change summary
0.9	15-09-2026	Team 14	Initial validation planning inside Deliverable 2.
1.0	03-10-2026	Team 14	Full test plan: strategy, security validation (§5.1), environment and seed data, schedule, 72 test cases, traceability to the SRS.

1. Introduction

Purpose. This plan says how the Railway Reservation System (RRS) v1.0 will be tested: objectives, scope, strategy, security validation, environment, schedule, responsibilities, the test cases and their traceability to the SRS. It is the validation specification updated after the architecture and design (Part B). **Scope.** The five functional areas, security requirements SEC-001 to SEC-006 and non-functional requirements NFR-001 to NFR-007. Payment settlement, live railway integration and third-party library internals are excluded (§4). **References.** RRS SRS v1.0, RRS SAD v1.0, the PES University Jackfruit guidelines and Level 3 submission guide, OWASP ASVS, IEEE Std 829. **Abbreviations.** TC test case; UT / IT / ST unit / integration / system test; P95 95th percentile; ASan / UBSan AddressSanitizer / UndefinedBehaviorSanitizer; D+n the n-th day after the execution date. **Status.** The test cases are designed; the Actual / Result column is filled during execution in sprints 2–6, and no case is marked Pass before it has been run.

2. Test items

Item	Description	Owner	Source directory
Authentication module (C2)	Registration, login, sessions, profile, lock-out	Bhadra R S	auth/
Train and schedule module (C3)	Search, route, schedule, fare quotes	Kshitij Prabhu	train/
Booking module (C4)	Availability, fare, seat allocation, PNR, history	Catherine D	booking/
Cancellation, waitlist and admin modules (C5)	Cancel, refund, promotion, status; train/coach admin; staff operations; reports	K Pranav	cancel/, admin/
Security and audit services (C6)	AccessControl, InputValidator, AuditLogger, PasswordHasher	Shared	security/
Data access layer (C7) and database (C8)	Repositories, transactions, rrs.db, audit.log	Shared	dal/
Console UI (C1)	Role-based menus and screens	All	ui/
Complete application	rrs executable, build scripts, CI pipeline	All	CMakeLists.txt, Jenkinsfile

3. Features to be tested

Each feature is mapped to SRS requirement IDs and to the number of designed test cases; the per-requirement mapping is in §13.3.

Feature	SRS requirements	Priority	Owner	Levels used	Test cases
F1 Registration, login, profile	FR-001 – FR-004	High	M1 Bhadra R S	Unit, Integration, System	10
F2 Train search, schedule, route, fare	FR-005 – FR-008	High	M2 Kshitij Prabhu	Unit, Integration, System	10
F3 Availability, booking, fare, PNR, history	FR-009 – FR-015	High	M3 Catherine D	Unit, Integration, System	14
F4 Cancellation, refund, waitlist, status	FR-016 – FR-020	High	M4 K Pranav	Unit, Integration, System	12

Feature	SRS requirements	Priority	Owner	Levels used	Test cases
F5 Administration, staff, reports	FR-021 – FR-025	High / Medium	M4 K Pranav	Unit, Integration, System	10
Security	SEC-001 – SEC-006	High	Shared	Unit, Integration, Security	8
Non-functional	NFR-001 – NFR-007	High / Medium	Shared	Performance, reliability, usability, maintainability, portability	8
Total	38 requirements				72

4. Features not to be tested

Feature / area	Reason
Payment-gateway settlement	Out of scope (SRS §1.2); only fare calculation and acceptance are tested
Live railway infrastructure, signalling, live running status	Not integrated; the system uses its own seeded data
Third-party library internals (SQLite, GoogleTest, the PBKDF2 implementation)	Assumed correct; only their use is tested
Graphical / web / mobile front-ends	None exist; the console is the only interface
Partial (single-passenger) cancellation of a multi-traveller PNR	Not a requirement; documented as a limitation (SAD §4.6)
Concurrent multi-terminal access and network behaviour	Single-process standalone application; would need a new design and threat model
Operating-system and compiler defects	Out of the team's control; portability is tested on two toolchains (PORT-01)

5. Test approach / strategy

Test levels

Level	What is tested	How	Responsible
Unit (UT)	One class or function in isolation – validators, FareCalculator, SeatAllocator, PnrGenerator, RefundCalculator, WaitlistManager, Session	GoogleTest with in-memory repository fakes; no file or database access	Module owner
Integration (IT)	A service together with the real SQLite repositories and the security services	GoogleTest against a temporary rrs.db created from the seed script	Module owner + peer reviewer
System (ST)	Complete use cases through the console menus, per role	Scripted stdin/stdout runs (shell or Python driver) and manual walkthroughs	Test lead + owners
Security (SEC-T)	SEC-001 – SEC-006 including penetration-style attempts	Manual and scripted tests, fuzzing, sanitizers, static analysis (§5.1)	M1 lead, all
Non-functional	Performance, reliability, usability, maintainability, portability	Timed scripts on the 20,000-reservation database, soak run, user walkthrough, gcov, two builds	Shared
Acceptance	All High-priority requirements and the exit criteria of SRS §6.2	Demonstration to faculty with the executed RTM	All

Test types and design techniques

Type	Purpose	Design technique
Functional	Each requirement behaves as specified	Equivalence partitioning of inputs; use-case scenarios
Negative / boundary	Invalid, empty, extreme values are rejected cleanly	Boundary-value analysis (age 0/1/120/121, 48 h / 12 h refund edges, 1/6/7 travellers)
State-based	Ticket-status transitions and waitlist order	State-transition testing from SM-01; decision table for the status decision
Regression	No earlier behaviour broken by a change	The whole automated suite runs on every push in CI
Performance / reliability	Response time, soak, retry safety	Load script, fault injection after COMMIT
Usability	Booking in ≤ 6 steps	Task-based walkthrough with 5 first-time users
Security	Confidentiality, access control, input safety, audit	Threat-based tests from the STRIDE model (SAD §3.9)

- GoogleTest and CTest run unit and integration tests on every push and pull request in the Jenkins pipeline (build → tests → coverage → static analysis).
- gcov/lcov measure coverage (targets: line ≥ 80 %, branch ≥ 70 % on the business layer, NFR-005).
- AddressSanitizer, UBSan, Valgrind, cppcheck, clang-tidy and SonarQube provide dynamic and static analysis.
- A Python driver feeds menu input to the console application for system tests and the soak test; an SQL seed script rebuilds the same test database for every run.

Entry criteria	Exit criteria
Stable build compiles with -Wall -Wextra -Werror on the CI image Interface headers of the sprint are frozen Seed data script and test accounts available Previous sprint's regression suite is green	100 % of planned test cases of the sprint executed 0 open critical and 0 open major defects at release Every High-priority requirement has a passing test (RTM column 'St.' = A) Coverage targets of NFR-005 met; sanitizers and cppcheck clean

Defect severity and handling

Severity	Definition	Target fix
Critical	Crash, data loss, duplicate seat/PNR, security bypass	Before next test cycle
Major	Requirement not met, wrong fare/refund/status, no workaround	Same sprint
Minor	Cosmetic or wording issue, workaround exists	Next sprint

Defects are logged as GitHub Issues labelled with severity, module and test-case id, linked to the fixing pull request and to the project board.

5.1 Security validation

Security validation comes from the security requirements (SRS §5.1) and the STRIDE model (SAD §3.9), and combines review, static analysis, dynamic analysis and penetration-style tests.

Requirement	Validation method	Tools / technique	Pass criterion	Test cases
SEC-001 No plaintext credentials	Storage inspection; code review of hashing	SQLite dump + grep; two accounts, same password	No password text anywhere; hashes differ; salt per row	UT-AUTH-02, SEC-T01
SEC-002 Role-based access	Authorisation matrix test; IDOR attempts	Every operation × every role (SAD §3.9.3)	Only permitted cells succeed; others E1006; no data change	IT-CAN-02, SEC-T02, SEC-T05, SEC-T08
SEC-003 Input validation	Negative tests; fuzzing; injection	1,000-char, control, SQL-meta, %, UTF-8 corpus; ASan + UBSan	Clean error, no crash, sanitizers silent, DB intact	UT-AUTH-01, UT-TRN-01, UT-BKG-02, UT-CAN-05, SEC-T03, SEC-T08
SEC-004 Session / lock-out	Time-based and tamper tests	Test-clock hook; edited session role	Expired/tampered session rejected; lock after 5 failures	UT-AUTH-04, ST-AUTH-02, SEC-T04, SEC-T06
SEC-005 Data minimisation	Display, log and retention checks	Log grep; retention routine on 91- and 89-day-old data	Masked ID, none in logs, cleared after 90 days	SEC-T07
SEC-006 Audit logging	Event-trail inspection	Scripted failed logins, forbidden calls, admin changes	Every event present with timestamp, actor, type; no secrets	SEC-T05, SEC-T06

Penetration-style tests

Attack	Technique	Expected defence
Authentication bypass	Login with ' OR '1'='1, empty and over-long identifiers	Bound parameters, validator – login fails
Brute force	Repeated wrong passwords against one account	Lock-out after 5 failures, audit entries
Privilege escalation	Passenger session calls admin/staff services; hidden menu numbers; edited role	RBAC E1006; role only from session
Insecure direct object reference	Passenger asks for or cancels another passenger's PNR	Ownership check E1006 + audit
Memory-safety faults	Oversized and malformed inputs, format specifiers	Length limits; sanitizers clean
Data tampering	Edit rrs.db constraints (duplicate PNR, negative age) and restart	CHECK/UNIQUE constraints; integrity check on start-up

Static security analysis (cppcheck, clang-tidy, SonarQube hotspots) runs in CI; findings of severity Major or higher must be fixed or justified before release.

6. Test environment

Area	Details
Hardware	Standard laptop/desktop (≥ 4 GB RAM) per team member; no special hardware
Operating systems	Ubuntu 22.04 LTS (primary, CI image) and Windows 11 with MinGW-w64
Software	GCC/G++ 11+, CMake, SQLite 3 (vendored), GoogleTest, gcov/lcov, Valgrind, cppcheck, clang-tidy, Python 3 (test driver)
Tools	GitHub Issues + Projects (defect and sprint tracking), Jenkins in Docker (CI), SonarQube
Test data	Seed script creates the data below; a large-data script generates 200 trains, 40 stations and 20,000 reservations for PERF-01/02
Configuration	Defaults of SRS Table 2.7-B; test-clock hook for expiry and time-to-departure tests; fault-injection hook for REL-02

Seed data

Stations	Code
Bengaluru City · Katpadi Jn · Chennai Central · Mysuru Jn · Secunderabad Jn	SBC · KPD · MAS · MYS · SC

Train	Name	Route (km from origin) and times	Classes (coaches $\times$ seats, RAC quota, WL capacity)
20001	Bengaluru–Chennai Express	SBC 06:00 (0) $\rightarrow$ KPD 09:10 (210) $\rightarrow$ MAS 11:30 (350)	SL 4 $\times$ 72 · 3A 2 $\times$ 64; RAC 0; WL 20
20002	Chennai–Bengaluru Express	MAS 14:00 (0) $\rightarrow$ KPD 16:10 (140) $\rightarrow$ SBC 20:30 (350)	SL 4 $\times$ 72 · 3A 2 $\times$ 64; RAC 0; WL 20
20003	Bengaluru–Mysuru Intercity (small-capacity test train)	SBC 07:00 (0) $\rightarrow$ MYS 10:00 (140)	SL 1 $\times$ 2, RAC 0, WL 2 · 3A 1 $\times$ 2, RAC 1, WL 2
20004	Bengaluru–Hyderabad Express	SBC 18:00 (0) $\rightarrow$ SC 06:30 next day (570)	SL 4 $\times$ 72 · 3A 2 $\times$ 64; RAC 0; WL 20

Account	Role	Login	Purpose
P1 Asha Rao	PASSENGER	9876500001	Main passenger (password Test@1234 – test data only)
P2 Ravi Kumar	PASSENGER	9876500002	Second passenger for ownership and IDOR tests
S1	STAFF	staff01	Reservation staff
A1	ADMIN	admin01	System administrator
O1	OPS	ops01	Railway operations

Fares used in the tests (₹1.00 per km $\times$ class multiplier): 350 km $\rightarrow$ SL ₹350.00, 3A ₹875.00; 210 km $\rightarrow$ SL ₹210.00, 3A ₹525.00; 140 km $\rightarrow$ SL ₹140.00; 570 km $\rightarrow$ 3A ₹1,425.00.

7. Test schedule

Follows the six sprints of the project plan (SRS Appendix B). Dates are indicative and re-planned at each sprint review.

Milestone	Sprint	Indicative date	Owner
Test plan and test-case design baseline (this document)	1	03-10-2026	All
Environment set-up: CMake, Docker, Jenkins, seed scripts, GitHub board	2	05 – 09-10-2026	K Pranav, Kshitij Prabhu
Unit and integration tests – F1 and F2 executed	2	by 16-10-2026	Bhadra R S, Kshitij Prabhu
Unit and integration tests – F3 executed; coverage report v1	3	by 30-10-2026	Catherine D
System tests – booking end to end (ST-BKG)	4	by 13-11-2026	Catherine D, all
F4 and F5 tests, security tests SEC-T01 – T08	5	by 27-11-2026	K Pranav, Bhadra R S
Performance, soak, usability, portability tests; final regression	6	by 04-12-2026	All
Test summary report and demonstration	6	by 11-12-2026	All

8. Test deliverables

- This test plan and the test-case catalogue (§13).
- Automated test sources (tests/unit, integration, system) and fakes in the Git repository.
- Seed and large-data scripts and test accounts.
- Execution logs with Actual Result and Pass/Fail per test case.
- Unit-test and coverage reports; static-analysis and sanitizer reports.

- Defect reports (GitHub Issues) and the defect summary.
- Final test summary report with the executed RTM (status column of SRS §8 updated to A).

9. Roles and responsibilities

Each member tests the module he or she develops and reviews a peer's tests.

Role	Name	Responsibility
Test lead and RTM owner	Kshitij Prabhu	Maintains this plan and the RTM; M2 unit and integration tests; coordinates system tests; test summary report
Test engineer – security	Bhadra R S	M1 tests (UT/IT/ST-AUTH); owns SEC-T01 – T08; static analysis and sanitizer runs
Test engineer – performance	Catherine D	M3 tests (UT/IT/ST-BKG); PERF-01/02, REL-01/02 and the large-data script
Test engineer – CI and coverage	K Pranav	M4 tests (CAN, ADM, STF, RPT); Jenkins pipeline, gcov/lcov, MNT-01, TST-01, PORT-01
Developer – defect fixing	Module owner	Fixes defects of own module within the target times; reviews peer pull requests
Product owner / sign-off	Course faculty	Reviews the RTM and the demonstration

Peer-review ring (tests and code)	Reviewer
Bhadra R S → reviewed by	Catherine D
Catherine D → reviewed by	Bhadra R S
Kshitij Prabhu → reviewed by	K Pranav
K Pranav → reviewed by	Kshitij Prabhu

10. Risks and mitigation

Risk	Mitigation
Delay in stable builds from a module owner	Interface stubs and fakes let others test early; smoke test in CI on every push
Test database state leaks between runs	Every test creates a fresh temporary rrs.db from the seed script
Time-dependent tests are flaky (expiry, refund slabs)	Injectable clock; no test reads the real time
Windows toolchain behaves differently from Linux	Build and run the unit suite on both platforms from Sprint 2
Large-data performance test too slow to run often	Generated once and cached; run before each sprint review
Unequal testing effort across members	Each member owns tests of one module; contribution visible in per-member commits and pull requests

11. Assumptions and dependencies

- The interface headers of SAD §4.3 are frozen per sprint; tests are written against them.
- SQLite, GoogleTest and the CI image are available to all team members.
- The system clock and the test-clock hook behave as specified.
- The seed and large-data scripts are maintained together with the schema.
- Five volunteers for the usability walkthrough are available in sprint 6.

12. Suspension and resumption criteria

Suspend testing if	Resume testing when
The build does not compile or the CI image is unavailable for more than 4 hours More than 30 % of the planned cases are blocked by one defect More than 3 critical defects are open The seed database cannot be created	Blocking defects are fixed and verified by a smoke test CI is restored Critical defects are reduced to zero The seed script runs successfully

13. Test case management and traceability

13.1 Naming and format

Test-case ids look like **LEVEL-MODULE-nn**: UT / IT / ST for unit, integration and system tests (modules AUTH, TRN, BKG, CAN, ADM, STF, RPT), SEC-Tnn for security tests, and PERF, REL, USB, MNT, TST, PORT for non-functional tests. Each case records requirement, level, description, precondition, steps, test data, expected result and Actual / Result ("Not run" in this version). D+n means n days after the execution date; account and train names refer to the seed data in §6. **The 72 designed cases**: 29 integration, 20 unit, 8 security, 7 system, 2 performance, 2 reliability, 2 maintainability, 1 usability, 1 portability.

13.2 Test-case catalogue

A Registration, login and profile (M1)

Test case ID	Requirement	Level	Description	Precondition	Steps	Test data	Expected result	Actual / Result
UT-AUTH-01	FR-001, SEC-003	Unit	Validate registration identifiers and password policy	InputValidator available	1) Call validateMobile / validateEmail / validatePassword with each value	Valid: 9876500001, asha@example.com, Test@1234. Invalid: 12345, 98765000012, asha@, empty, abc12, abcdefgh, 12345678	Valid values accepted; each invalid value rejected with ERR-VALIDATION (E1001) naming the field.	Not run
UT-AUTH-02	SEC-001, FR-001	Unit	Password hashing is salted and not reversible	PasswordHasher available	1) hash() the same password with two random salts; 2) verify() with the right and a wrong password	Test@1234; two 16-byte salts	Hash is 64 hex characters and differs from the plaintext; different salts give different hashes; verify() is true only for the correct password.	Not run
IT-AUTH-01	FR-001	Integration	Register a new passenger	Empty user table	1) registerPassenger(new mobile, password); 2) inspect the database	9876500001, Asha Rao, Test@1234	Result ok; exactly one USER_ACCOUNT row, role PASSENGER, unique account_id; the password column holds a hash, not the password.	Not run
IT-AUTH-02	FR-004	Integration	Duplicate registration is rejected	Account with mobile 9876500001 and e-mail asha@example.com exists	1) Register again with the same mobile; 2) register with the same e-mail and a new mobile	as stated	Both attempts return ERR-DUP-USER (E1002); the account count stays 1.	Not run
UT-AUTH-03	FR-002	Unit	Successful login creates a role-bound session	Account P1 exists	1) login(9876500001, Test@1234)	P1 credentials	Result ok; Session.role = PASSENGER; expiresAt = now + 15 min; failedAttempts reset to 0.	Not run
UT-AUTH-04	FR-002, SEC-004	Unit	Failed login returns a generic error	Account P1 exists	1) login with a wrong password; 2) login with an unknown id	P1 + Wrong@123; 9000000000	Both return ERR-AUTH-INVALID (E1003) with the identical message (no user enumeration); no session; failedAttempts incremented for the known account.	Not run
IT-AUTH-03	FR-003	Integration	Create and update the passenger profile	Logged in as P1	1) updateProfile(full data); 2) getProfile()	Asha Rao, 28, F, ID DL-KA012026001	Stored values are returned unchanged; the ID number is displayed masked (last 4 characters only).	Not run
UT-AUTH-05	FR-003	Unit	Profile boundary validation	Logged in as P1	1) Submit ages 0, 1, 120, 121; 2) submit names of length 0, 1, 100, 101	boundary values as listed	Ages 1 and 120 and names of length 1 and 100 accepted; the other values rejected with E1001.	Not run
ST-AUTH-01	FR-001, FR-002, FR-003	System	Register, log in and update profile through the console	Fresh database; application started	1) Register; 2) Login; 3) My Profile → edit; 4) Logout and login again; 5) View profile	new passenger data	Each step succeeds; the second login shows the updated profile; all messages follow the standard format.	Not run
ST-AUTH-02	FR-002, SEC-004	System	Session expires after 15 minutes of inactivity	Logged in as P1; test-clock hook available	1) Advance the clock by 16 minutes; 2) choose Book Ticket	clock + 16 min	Operation rejected with ERR-SESSION-EXPIRED (E1005); user returned to the login screen; nothing is booked.	Not run

B Train search, schedule, route and fare (M2)

Test case ID	Requirement	Level	Description	Precondition	Steps	Test data	Expected result	Actual / Result
UT-TRN-01	FR-005, SEC-003	Unit	Validate search input	InputValidator available	1) Validate each (source, destination, date) combination	SBC-SBC; SBC-XXX; SBC-MAS yesterday; SBC-MAS today+121; SBC-MAS today; SBC-MAS today+120	The first four are rejected (E1001 with reason); the last two are accepted.	Not run
UT-TRN-02	FR-005	Unit	Route-direction matching	Seed route of train 20001	1) servesLeg(20001, SBC→MAS), (SBC→KPD), (KPD→MAS), (MAS→SBC)	seed data	true, true, true, false.	Not run
IT-TRN-01	FR-005, FR-006	Integration	Search returns exactly the matching trains	Seed data loaded	1) searchTrains(SBC→MAS, today+7)	SBC, MAS	Only train 20001 is listed; 20002, 20003 and 20004 are absent.	Not run
IT-TRN-02	FR-006	Integration	Result row contains all required fields	As IT-TRN-01	1) Inspect the row for train 20001	-	Number 20001, name, departure 06:00, arrival 11:30, duration 05:30.	Not run
UT-TRN-03	FR-006	Unit	Duration across midnight	Seed train 20004	1) duration(18:00, 06:30 next day)	-	12:30.	Not run
IT-TRN-03	FR-005	Integration	Search with no matching train	Seed data loaded	1) searchTrains(MYS→SC, today+7)	MYS, SC	Empty list; screen says "No trains found for this route and date."; no error code and no crash.	Not run
IT-TRN-04	FR-007	Integration	View schedule and route	Seed data loaded	1) getRoute(20001)	20001	Three stops in order SBC, KPD, MAS with arrival/departure times and cumulative km 0, 210, 350.	Not run
UT-TRN-04	FR-008, FR-011	Unit	Fare per class for one traveller	FareCalculator available	1) totalFare(km, class, 1) for each pair	350 km SL and 3A; 210 km SL and 3A; 570 km 3A	350.00; 875.00; 210.00; 525.00; 1,425.00.	Not run
IT-TRN-05	FR-008	Integration	Fare quotes list every configured class	Seed data loaded	1) getFareQuotes(20001, SBC, MAS)	-	Two quotes: Sleeper 350.00 and AC 3-Tier 875.00; no quote for a class that is not configured.	Not run
ST-TRN-01	FR-005, FR-006, FR-007, FR-008	System	Search, route and fare through the console	Application started	1) Search SBC→MAS; 2) select 20001; 3) view route; 4) view fares	-	All screens show correct data; the main menu is reachable from every screen.	Not run

C Availability, booking, fare, PNR and history (M3)

Test case ID	Requirement	Level	Description	Precondition	Steps	Test data	Expected result	Actual / Result
UT-BKG-01	FR-011	Unit	Total fare calculation	FareCalculator available	1) totalFare(km, class, travellers)	350 km 3A ×2; 350 km SL ×1; 350 km SL ×6; 33 km 3A ×1	1,750.00; 350.00; 2,100.00; 82.50.	Not run
UT-BKG-02	FR-010, SEC-003	Unit	Traveller-list validation	InputValidator available	1) Validate lists of 0, 1, 6, 7 travellers; 2) validate individual invalid fields	age 0; gender X; empty name; empty ID proof	Lists of 1 and 6 accepted; 0, 7 and every invalid field rejected with a field-level message.	Not run
UT-BKG-03	FR-014	Unit	PNR generation	PnrGenerator with exists() stub	1) Generate 10,000 PNRs; 2) force a collision on the first draw	-	Every PNR has 10 digits with first digit 1-9 and no duplicates; after a forced collision a new value is drawn (≤ 5 retries).	Not run
UT-BKG-04	FR-013	Unit	Seat allocation order	SeatAllocator, capacity 2	1) allocate 1 (none occupied); 2) allocate 1 (S1-01 occupied); 3) allocate 1 (both occupied)	-	S1-01; S1-02; capacity-exhausted result.	Not run
UT-BKG-05	FR-009	Unit	Availability computation	Availability logic with fake repository	1) capacity 2, 1 confirmed; 2) capacity 2, 1 confirmed + 1 cancelled	-	1 seat free; 2 seats free (cancelled seats are released).	Not run
IT-BKG-01	FR-009	Integration	Displayed availability equals the database	Train 20003 SL, date D+7, no bookings	1) checkAvailability; 2) book 1 seat; 3) checkAvailability	-	2 free, RAC 0, WL 0/2; after the booking 1 free.	Not run

Test case ID	Requirement	Level	Description	Precondition	Steps	Test data	Expected result	Actual / Result
IT-BKG-02	FR-012, FR-013, FR-014	Integration	Successful single booking	Logged in as P1; 20003 SL empty	1) createBooking SBC→MYS, 1 traveller, Sleeper	Asha Rao, 28, F, ID DL-KA012026001	Status CONFIRMED; seat S1-01; 10-digit PNR; fare 140.00 stored; one RESERVATION and one TRAVELLER row.	Not run
IT-BKG-03	FR-012, FR-010	Integration	Invalid traveller detail blocks the booking	Logged in as P1	1) createBooking with age 0	age 0	ERR-VALIDATION (E1001); no reservation row; availability unchanged.	Not run
IT-BKG-04	FR-012	Integration	Fare not accepted – no booking	Logged in as P1	1) Reach the fare screen; 2) answer N	N	No reservation is created; availability unchanged; user returns to the menu.	Not run
IT-BKG-05	FR-013	Integration	No seat is allocated twice	20003 SL has exactly one free seat	1) Submit two bookings back-to-back for the last seat	P1, P2	First ticket CONFIRMED with the seat; second WAITLISTED (WL1); no seat number appears twice among active tickets.	Not run
IT-BKG-06	NFR-004, FR-014	Integration	Duplicate submission is idempotent	Logged in as P1	1) createBooking with request_id R; 2) repeat with the same R	request_id R	The second call returns the same PNR; exactly one reservation exists.	Not run
IT-BKG-07	FR-015	Integration	Booking history shows own reservations only	P1 has 1 past and 2 upcoming reservations; P2 has 1	1) getBookingHistory(P1)	seeded	Three reservations of P1 ordered by journey date and labelled Upcoming/Past; P2's reservation is absent.	Not run
ST-BKG-01	FR-009 – FR-014, NFR-003	System	End-to-end booking through the console	P1 registered	1) Login; 2) Search; 3) Select train and class; 4) Enter traveller; 5) Accept fare; 6) View ticket	SBC→MYS, Sleeper, 1 traveller	Ticket with PNR, status, seat and fare is displayed; counted menu steps ≤ 6.	Not run
ST-BKG-02	FR-010, FR-011, FR-012	System	Multi-traveller booking	Logged in as P1; 20001 3A available	1) Book 3 travellers, AC 3-Tier, SBC→MAS	3 travellers	One PNR; three distinct seats; total fare 2,625.00 (875.00 × 3).	Not run

D Cancellation, waitlist and status (M4)

Test case ID	Requirement	Level	Description	Precondition	Steps	Test data	Expected result	Actual / Result
UT-CAN-01	FR-017, BR-06	Unit	Refund slabs and boundaries	RefundCalculator available	1) refund(1000.00, hoursToDeparture, status) for each case	72 h; 48 h 00 m; 47 h 59 m; 12 h 00 m; 11 h 59 m; WAITLISTED at 5 h	900.00; 900.00; 750.00; 750.00; 500.00; 1,000.00.	Not run
UT-CAN-02	FR-016	Unit	Cancellation eligibility	Reservation fixtures	1) isCancellable for a future journey, a departed journey, an already cancelled ticket	-	Allowed; ERR-CANCEL-DEPARTED (E4003); ERR-ALREADY-CANCELLED (E4002).	Not run
IT-CAN-01	FR-016, FR-017	Integration	Cancel a confirmed ticket	P1 holds a CONFIRMED ticket (fare 140.00) for D+3	1) cancelTicket(P1, pnr)	PNR of that ticket	Status CANCELLED; seat released (availability +1); refund 126.00 (90 %); audit event written.	Not run
IT-CAN-02	FR-016, SEC-002	Integration	Cancel with a foreign or unknown PNR	P1 and P2 hold tickets	1) P2 cancels P1's PNR; 2) P1 cancels 9999999999	-	ERR-FORBIDDEN (E1006) plus audit entry; ERR-PNR-NOT-FOUND (E4001); no state change.	Not run
UT-CAN-03	FR-018, BR-01	Unit	Status decision when capacity is exhausted	20003 SL: 2 seats, RAC 0, WL capacity 2	1) Book 5 single-traveller tickets in sequence	-	#1 and #2 CONFIRMED; #3 WL1; #4 WL2; #5 rejected with ERR-NO-CAPACITY (E3001).	Not run
IT-CAN-03	FR-019	Integration	Waitlist promotion on cancellation	State left by UT-CAN-03 (#1, #2 confirmed; #3 WL1; #4 WL2)	1) Cancel ticket #1	-	#3 becomes CONFIRMED with the freed seat; #4 moves WL2 → WL1; all in one transaction.	Not run
UT-CAN-04	FR-019, BR-07	Unit	Strict-order promotion	WL1 holds 2 travellers, WL2 holds 1; one seat is freed	1) promote()	-	Nobody is promoted: WL1 does not fit and WL2 may not skip the head of the queue.	Not run
IT-CAN-04	FR-018, FR-017	Integration	Cancel a waitlisted ticket	WL1 and WL2 exist	1) Cancel WL1	-	WL1 CANCELLED with 100 % refund; WL2 becomes WL1.	Not run
IT-CAN-05	FR-018, FR-019, FR-020	Integration	RAC path	20003 3A: 2 seats, RAC quota 1, WL capacity 2	1) Book 4 tickets; 2) cancel #1	-	#1, #2 CONFIRMED; #3 RAC; #4 WL1. After cancelling #1: #3 CONFIRMED and #4 RAC.	Not run

Test case ID	Requirement	Level	Description	Precondition	Steps	Test data	Expected result	Actual / Result
IT-CAN-06	FR-020	Integration	PNR status text	Tickets exist in states Confirmed, RAC, WL2, Cancelled	1) getTicketStatus for each PNR	-	"Confirmed", "RAC", "Waitlisted - position 2", "Cancelled".	Not run
UT-CAN-05	FR-020, SEC-003	Unit	PNR format validation	InputValidator available	1) Validate each PNR string	12AB; 123456789; 12345678901; ' OR 1=1; 1234567890	The first four are rejected before any database access (E1001); the last is accepted.	Not run
ST-CAN-01	FR-016 – FR-020	System	End-to-end book, cancel and status	P1 logged in	1) Book; 2) PNR Status; 3) Cancel; 4) PNR Status	-	Status moves Confirmed → Cancelled; the refund amount is shown; the seat is available again in search.	Not run

E Administration, staff operations and reports (M4)

Test case ID	Requirement	Level	Description	Precondition	Steps	Test data	Expected result	Actual / Result
IT-ADM-01	FR-021	Integration	Admin adds train, route and schedule	Logged in as A1	1) addTrain 20099 (stops SBC 0 km, MYS 140 km) with a daily schedule; 2) search SBC→MYS	20099	Train 20099 appears in the search for its route and date.	Not run
IT-ADM-02	FR-021	Integration	Admin updates a train	20099 exists with one reservation	1) update departure time; 2) search; 3) check the reservation	new departure 08:15	Search shows 08:15; the existing PNR is unchanged.	Not run
IT-ADM-03	FR-021, BR-04	Integration	Remove a train with and without active bookings	20099 has an active booking; 20098 has none	1) removeTrain(20099); 2) removeTrain(20098)	-	20099: ERR-TRAIN-IN-USE (E5001) and still searchable; 20098 deactivated and no longer searchable.	Not run
UT-ADM-01	FR-021	Unit	Route validation rules	Route validator	1) Validate five candidate routes	1 stop; duplicate station; non-increasing km; non-increasing times; valid 3-stop route	The first four are rejected with a reason; the valid route is accepted.	Not run
IT-ADM-04	FR-022	Integration	Configure coaches and capacity	20099 SL configured as 1 coach × 2 seats	1) set SL to 2 coaches × 3 seats; 2) check availability; 3) with 4 seats booked set 1 × 3	-	Capacity 6 is shown; the reduction below the booked seats is rejected with ERR-CAPACITY-BELOW-BOOKED (E5002).	Not run
IT-STF-01	FR-023	Integration	Staff books on behalf of a passenger	Logged in as S1	1) bookOnBehalf(passenger details)	valid traveller data	PNR generated; same validation as the passenger path; booked_by = S1.	Not run
IT-STF-02	FR-024, BR-03	Integration	Staff corrects and cancels a reservation	Logged in as S1; reservation exists	1) correct a traveller name; 2) cancel the reservation	-	Changes saved; audit entries contain actor, time, old and new values.	Not run
IT-RPT-01	FR-025	Integration	Booking statistics	10 reservations seeded: 6 CONFIRMED, 2 WAITLISTED, 2 CANCELLED	1) bookingStatistics(date range)	-	Report shows 6 / 2 / 2 and a total of 10, equal to the SQL COUNT per status.	Not run
IT-RPT-02	FR-025	Integration	Seat occupancy and train-wise counts	20003 SL (capacity 2) fully booked	1) seatOccupancy(20003, date); 2) trainWiseCounts()	-	Occupancy 100 %; train-wise counts match the database.	Not run
ST-ADM-01	FR-021, FR-023, FR-025	System	End-to-end admin, staff and operations flow	Seed users A1, S1, O1	1) A1 adds a train; 2) S1 books on it; 3) O1 opens the report	-	The report includes the new booking; each role sees only its own menu.	Not run

F Security validation

Test case ID	Requirement	Level	Description	Precondition	Steps	Test data	Expected result	Actual / Result
SEC-T01	SEC-001	Security	No plaintext credentials	Accounts registered with Test@1234	1) Dump rrs.db and audit.log; 2) search both for the password; 3) compare the hashes of two accounts	-	No match; the hashes differ; a salt exists per row.	Not run
SEC-T02	SEC-002, FR-024	Security	Role-based access matrix	Sessions for all four roles	1) Invoke every operation as every role (menu and direct service call)	permission matrix (SAD §3.9.3)	Allowed only where the matrix says so; all other calls return ERR-FORBIDDEN (E1006); no data changes.	Not run
SEC-T03	SEC-003	Security	Input fuzzing	Build with AddressSanitizer + UBSan	1) Submit 1,000-char strings, control characters, ' OR 1=1 --, %s%n and UTF-8 text to every text field	fuzz corpus	Clean validation error or safe storage; no crash; sanitizers report nothing; database integrity check passes.	Not run
SEC-T04	SEC-004	Security	Expired and tampered sessions	Session for P1	1) Use it after 16 minutes idle; 2) change the role field to ADMIN; 3) use an empty session	-	Each case is rejected with ERR-SESSION-EXPIRED (E1005) or ERR-FORBIDDEN; no privileged action runs.	Not run
SEC-T05	SEC-006, SEC-002	Security	Security events are audited	Clean audit log	1) 3 failed logins; 2) 1 unauthorised admin call; 3) 1 train update by admin	-	audit.log gains 5 entries with timestamp, event type and actor id; no password text appears.	Not run
SEC-T06	SEC-004, SEC-006	Security	Account lock-out after repeated failures	Account P1	1) 5 consecutive wrong passwords; 2) 6th attempt with the correct password; 3) wait 5 minutes and retry	-	Account locked after the 5th failure (E1004); the correct password is rejected while locked and works after 5 minutes; both events are logged.	Not run
SEC-T07	SEC-005	Security	ID-proof masking and retention	Reservations with journey dates 91 and 89 days ago	1) View a ticket; 2) search the logs; 3) run the retention routine	-	ID shown as last 4 characters only and absent from logs; the 91-day-old ID is cleared, the 89-day-old one retained.	Not run
SEC-T08	SEC-002, SEC-003	Security	Penetration-style attempts	Running application	1) Log in with ' OR '1'='1; 2) P1 requests P2's PNR (IDOR); 3) tamper with hidden menu numbers	-	No authentication bypass; the IDOR attempt is rejected (E1006); hidden options answer "Invalid choice"; every attempt is audited.	Not run

G Non-functional requirements

Test case ID	Requirement	Level	Description	Precondition	Steps	Test data	Expected result	Actual / Result
PERF-01	NFR-001	Performance	Search response time	Database with 200 trains, 40 stations, 20,000 reservations	1) Run 100 sequential searches with varied routes and dates; 2) record the latency	-	At least 95 of 100 searches finish in ≤ 2.0 s (P50, P95 and maximum reported).	Not run
PERF-02	NFR-001, FR-009	Performance	Availability and booking latency	Same database as PERF-01	1) 50 availability queries; 2) 50 bookings	-	P95 ≤ 2.0 s for each operation.	Not run
REL-01	NFR-002	Reliability	Soak test	Freshly seeded database	1) Scripted run of 1,000 mixed operations (search, book, cancel, status)	-	No crash; ≥ 99 % of operations complete normally (expected business errors excluded); log reviewed.	Not run
REL-02	NFR-004	Reliability	Retry after a confirmation timeout	Fault-injection hook after COMMIT	1) Book and kill the process before the confirmation is shown; 2) restart; 3) retry with the same request token	-	Exactly one reservation exists; the retry returns the original PNR.	Not run
USB-01	NFR-003	Usability	First-time-user walkthrough	Five volunteers without training	1) Each volunteer completes one booking; an observer counts menu steps and errors	-	At least 4 of 5 complete unaided; every run needs ≤ 6 primary menu steps.	Not run
MNT-01	NFR-005	Maintainability	Modularity and coverage	Source tree and test run	1) Apply the review checklist (one header/source pair per component; UI never calls the data layer); 2) run gcov/lcov	-	Checklist 100 %; line coverage ≥ 80 % and branch coverage ≥ 70 % on the business layer.	Not run
TST-01	NFR-006	Maintainability	Helpers testable without I/O	Unit-test binary	1) Run the FareCalculator, SeatAllocator and PnrGenerator suites under strace -e openat	-	All tests pass; no open() of rrs.db or any other data file.	Not run

Test case ID	Requirement	Level	Description	Precondition	Steps	Test data	Expected result	Actual / Result
PORT-01	NFR-007	Portability	Cross-platform build	Linux GCC 11+ and Windows MinGW-w64	1) Build with -std=c++17 -Wall -Wextra -Werror; 2) run the unit suite on both	-	0 errors, 0 warnings; every test passes on both platforms.	Not run

13.3 Traceability – requirement to test cases

All 38 SRS requirements and the test cases that verify them. Architecture, design and code references are in SRS §8.

Req ID	Requirement (brief)	Test cases	No.
FR-001	Register with unique mobile / e-mail	UT-AUTH-01, UT-AUTH-02, IT-AUTH-01, ST-AUTH-01	4
FR-002	Log in with valid credentials	UT-AUTH-03, UT-AUTH-04, ST-AUTH-01, ST-AUTH-02	4
FR-003	Create and update profile	IT-AUTH-03, UT-AUTH-05, ST-AUTH-01	3
FR-004	Reject duplicate registration	IT-AUTH-02	1
FR-005	Search trains by source, destination, date	UT-TRN-01, UT-TRN-02, IT-TRN-01, IT-TRN-03, ST-TRN-01	5
FR-006	Show number, name, times, duration	IT-TRN-01, IT-TRN-02, UT-TRN-03, ST-TRN-01	4
FR-007	View schedule and route of a train	IT-TRN-04, ST-TRN-01	2
FR-008	Show fare for each class	UT-TRN-04, IT-TRN-05, ST-TRN-01	3
FR-009	Show real-time seat availability	UT-BKG-05, IT-BKG-01, ST-BKG-01, PERF-02	4
FR-010	Enter details of each traveller	UT-BKG-02, IT-BKG-03, ST-BKG-01, ST-BKG-02	4
FR-011	Calculate total fare before confirmation	UT-TRN-04, UT-BKG-01, ST-BKG-01, ST-BKG-02	4
FR-012	Confirm only valid, fare-accepted bookings	IT-BKG-02, IT-BKG-03, IT-BKG-04, ST-BKG-01, ST-BKG-02	5
FR-013	Allocate a seat / berth per passenger	UT-BKG-04, IT-BKG-02, IT-BKG-05, ST-BKG-01	4
FR-014	Generate a unique PNR	UT-BKG-03, IT-BKG-02, IT-BKG-06, ST-BKG-01	4
FR-015	Booking history for the passenger	IT-BKG-07	1
FR-016	Cancel a ticket before the journey	UT-CAN-02, IT-CAN-01, IT-CAN-02, ST-CAN-01	4
FR-017	Compute refund by cancellation policy	UT-CAN-01, IT-CAN-01, IT-CAN-04, ST-CAN-01	4
FR-018	Waitlist with visible position	UT-CAN-03, IT-CAN-04, IT-CAN-05, ST-CAN-01	4
FR-019	Automatic waitlist promotion	IT-CAN-03, UT-CAN-04, IT-CAN-05, ST-CAN-01	4
FR-020	Ticket status by PNR	IT-CAN-05, IT-CAN-06, UT-CAN-05, ST-CAN-01	4
FR-021	Admin: manage trains, routes, schedules	IT-ADM-01, IT-ADM-02, IT-ADM-03, UT-ADM-01, ST-ADM-01	5
FR-022	Admin: coach and seat configuration	IT-ADM-04	1
FR-023	Staff: book on behalf of a passenger	IT-STF-01, ST-ADM-01	2
FR-024	Staff / admin: correct or cancel records	IT-STF-02, SEC-T02	2
FR-025	Reports: bookings, occupancy, train-wise	IT-RPT-01, IT-RPT-02, ST-ADM-01	3
NFR-001	Search ≤ 2 s (P95)	PERF-01, PERF-02	2
NFR-002	Availability ≥ 99 %	REL-01	1
NFR-003	Booking in ≤ 6 menu steps	ST-BKG-01, USB-01	2
NFR-004	No duplicate reservation on retry	IT-BKG-06, REL-02	2
NFR-005	Modular, unit-tested business logic	MNT-01	1
NFR-006	Helpers testable without I/O	TST-01	1
NFR-007	Portable GCC build	PORT-01	1
SEC-001	No plaintext passwords	UT-AUTH-02, SEC-T01	2
SEC-002	Role-based access	IT-CAN-02, SEC-T02, SEC-T05, SEC-T08	4
SEC-003	Validate and sanitise input	UT-AUTH-01, UT-TRN-01, UT-BKG-02, UT-CAN-05, SEC-T03, SEC-T08	6
SEC-004	Reject invalid / expired session	UT-AUTH-04, ST-AUTH-02, SEC-T04, SEC-T06	4
SEC-005	Minimise retained ID data	SEC-T07	1
SEC-006	Log security events	SEC-T05, SEC-T06	2

13.4 Coverage

All 25 functional, 7 non-functional and 6 security requirements are covered by at least one test case. Most functional requirements are checked at several levels (unit, integration, system) so that a defect is caught early and again end to end.

14. Test metrics and reporting

Metric	Definition	Target at release
Test-case execution rate	Executed cases ÷ planned cases	100 %
Pass rate	Passed ÷ executed	100 % for High-priority requirements; ≥ 95 % overall before waivers
Requirement coverage	Requirements with ≥ 1 passing test ÷ 38	100 %
Code coverage	gcov line / branch on the business layer	≥ 80 % / ≥ 70 % (NFR-005)
Defect density	Defects per module and per 1,000 lines	Reported per sprint
Defect ageing	Days from report to verified fix	Critical ≤ 1 day, Major ≤ sprint
Open defects at release	By severity	0 critical, 0 major

Reporting: the CI result and test summary appear in every pull request; each sprint review shows execution status, coverage and a defect summary; the final test summary report carries the executed RTM.

15. Approvals

Role	Name	Signature / date
Test lead	Kshitij Prabhu	
QA - security	Bhadra R S	
QA - performance	Catherine D	
QA - CI and coverage	K Pranav	
Product owner / faculty	Dr. Sapna V M, PES University	